

A PROJECT REPORT ON

**Electron Eye: Activity Tracking and Visualisation
Application**

SUBMITTED TO THE SAVITRIBAI PHULE UNIVERSITY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE

OF

BACHELOR OF ENGINEERING (Computer Engineering)

SUBMITTED BY

Rutwik Malav
Shruti Maliye
Manav Mehta
Nikita More

Exam No: B400050487
Exam No: B400050491
Exam No: B400050496
Exam No: B400050510



DEPARTMENT OF COMPUTER ENGINEERING
Pune Institute of Computer Technology
Dhankawadi, Pune - 411043

SAVITRIBAI PHULE PUNE UNIVERSITY
2024-2025



CERTIFICATE

This is to certify that the project report entitled

Electron Eye: Activity Tracking and Visualisation Application

Submitted by

Rutwik Malav
Shruti Maliye
Manav Mehta
Nikita More

Exam No: B400050487
Exam No: B400050491
Exam No: B400050496
Exam No: B400050510

are bonafide students of this institute and the work has been carried out by them under the supervision of **Dr. K. C. Waghmare** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Bachelor of Engineering** (Computer Engineering).

Dr. K. C. Waghmare
Internal Guide
Dept. of Computer Engg.

Dr. G. V. Kale
Head
Dept. of Computer Engg.

Dr. S. T. Gandhe
Principal
Pune Institute of Computer Technology

Place: Pune
Date:

ACKNOWLEDGEMENT

*It gives us great pleasure in presenting the project report on ‘**Electron Eye: Activity Tracking and Visualisation Application**’.*

*We would like to take this opportunity to thank **Dr. K. C. Waghmare** for giving us all the help and guidance we needed. We are really grateful to her for her kind support. Her valuable suggestions were very helpful.*

*We are also grateful to **Dr. G. V. Kale**, Head of Computer Engineering Department, PICT for her indispensable support and suggestions throughout the project. We also express our gratitude to **Dr. A. R. Deshpande**, BE project coordinator, for her timely guidance and support.*

*We would also like to thank our respected **Dr. S. T. Gandhe**, Principal, PICT, for his constant encouragement, continued support, and invaluable feedback which motivated us throughout the duration of the project.*

*In the end our special thanks to our honorable **Dr. P. T. Kulkarni**, Director, PICT, for his continued support and valuable inputs during the ideation and implementation phase of the project. We also thank our laboratory assistants for their valuable help in setting up the workstations. We are deeply thankful for the platform and resources that PICT offers us, allowing us to undertake this valuable project.*

Rutwik Malav
Shruti Maliye
Manav Mehta
Nikita More
(B.E. Computer Engg.)

ABSTRACT

The proposed system will be a feature-rich desktop application made to assist lab managers by keeping track of computer applications, usage and system health. All access data will be combined and sent to a primary server, giving administrators insightful information about unauthorized actions such as USB usage, browser activity, and network access, usage trends that will enable them to better manage resources and increase lab efficiency. With the help of the technology, administrators will be able to provide predictive maintenance, which will let them foresee future system malfunctions or predict when certain systems will require repair based on usage and health indicators. Additionally, it will include sophisticated visualization, offering comprehensive reporting and analytics to support administrators in seeing patterns, allocating resources effectively, and coming to wise judgments. The user-friendly interface of this multitasking tool will guarantee smooth and productive lab activities.

Contents

1	Introduction	1
1.1	Overview	2
1.2	Motivation	2
1.3	Problem Definition and Objectives	2
1.4	Project Scope & Limitations	3
1.5	Methodologies of Problem solving	3
2	Literature Survey	5
3	Software Requirements Specification	8
3.1	Assumptions and Dependencies	9
3.2	Functional Requirements	9
3.2.1	System Feature 1: Real-Time System Monitoring	9
3.2.2	System Feature 2: Predictive Maintenance	9
3.2.3	System Feature 3: Reporting and Analytics	9
3.3	External Interface Requirements	10
3.3.1	User Interfaces	10
3.3.2	Hardware Interfaces	10
3.3.3	Software Interfaces	10
3.4	Nonfunctional Requirements	11
3.4.1	Performance Requirements	11
3.4.2	Safety Requirements & Security Requirements	11
3.5	System Requirements	11
3.5.1	Database Requirements	11
3.5.2	Software Requirements	11
3.5.3	Hardware Requirements	11
3.6	Analysis Models: Agile Methodology to be applied	11
4	System Design	13
4.1	System Architecture	14
4.2	Mathematical Model	15
4.3	Data Flow Diagrams	16
4.4	Use Case Diagram	18
4.5	Entity Relationship Diagram	19
4.6	Class Diagram	20
4.7	Sequence Diagram	21
4.8	Deployment Diagram	22

5	Project Plan	23
5.1	Project Estimate	24
5.1.1	Reconciled Estimates	24
5.1.2	Project Resources	24
5.2	Risk Management	25
5.2.1	Risk Identification	25
5.2.2	Risk Analysis	26
5.3	Project Schedule	27
5.3.1	Project Task Set	27
5.3.2	Timeline Chart	28
5.4	Team Organization	29
5.4.1	Team structure	29
5.4.2	Management reporting and communication	29
6	Project Implementation	30
6.1	Overview of Project Modules	31
6.2	Tools and Technologies Used	31
6.3	Algorithm Details	32
6.3.1	Algorithm 1: Data Collection Algorithm	32
6.3.2	Algorithm 2: USB Detection and Alert Algorithm	32
6.3.3	Algorithm 3: Real-time Notification and Dashboard Update	32
6.3.4	Algorithm 4: Data Transmission to Backend	32
6.3.5	Algorithm 5: Backend Data Storage and Management	32
6.3.6	Algorithm 6: Flagged Device Identification	33
6.3.7	Algorithm 7: Visualization Rendering Algorithm	33
6.3.8	Algorithm 8: Admin Panel PC Monitoring View	33
7	Software Testing	34
7.1	Types of Testing	35
7.2	Test cases & Test Results	35
8	Results	37
8.1	Outcomes	38
8.2	Screen Shots	38
9	Conclusions	53
9.1	Conclusion	54
9.2	Future Work	54
9.3	Applications	54
10	Appendix	56
10.1	Appendix A: Problem Statement Feasibility Assessment	57
10.2	Appendix B: Paper Publication Details	58
10.3	Appendix C: Plagiarism Report	59
11	References	60

List of Figures

4.1	System Architecture	14
4.2	DFD Level 0 – Context Level	16
4.3	DFD Level 1 – Detailed View	17
4.4	Use Case Diagram	18
4.5	Entity Relationship Diagram	19
4.6	Class Diagram	20
4.7	Sequence Diagram	21
4.8	Deployment Diagram	22
5.1	Summarized TimeLine Chart	28
8.1	Home Page	38
8.2	Register Page	39
8.3	Login Page	39
8.4	Chatbot	40
8.5	User interface for managing MAC Addresses, including adding new entries, viewing stored devices, and accessing further device analysis options. . .	42
8.6	Various Features for a particular MAC Address	42
8.7	Cheating Devices	43
8.8	Process Details	44
8.9	Network Details	45
8.10	Network Requests	46
8.11	Connected Devices	47
8.12	Alert when Pen drive is detected on a particular MAC Address	48
8.13	Browser History	49
8.14	System Health	50
8.15	Failure Alerts	51
8.16	Graphs	52
10.1	Plagiarism Report Snapshots for Various Project Sections	59

CHAPTER 1

INTRODUCTION

1.1 Overview

This project aims to develop a feature-rich desktop application to assist lab managers in effectively monitoring computer usage and system health. The application will collect and consolidate data from individual computers in a lab, sending it to a central server for analysis. Lab administrators can then access detailed reports on usage trends, allowing them to optimize resource management and improve overall efficiency. Additionally, the system will include advanced predictive maintenance features, which can foresee potential system failures or repairs based on real-time data from hardware health indicators, reducing unexpected downtimes. With comprehensive visualization tools, administrators can view detailed analytics and make data-driven decisions, ensuring optimal performance across all systems. The user-friendly interface guarantees a smooth experience, allowing even non-technical users to manage system data, monitor computer health, and allocate resources efficiently. By incorporating these capabilities, the system enhances lab operations, ensuring seamless workflows and increasing system reliability. Its multitasking features will also facilitate the coordination of multiple lab activities simultaneously, improving the efficiency of lab operations, reducing costs, and extending the life span of the systems under management.

1.2 Motivation

The motivation for developing this system arises from the increasing complexity of managing computer labs, which are integral to modern education, research, and organizational operations. Lab administrators are tasked with ensuring optimal system performance, efficient use of resources, and timely maintenance schedules. However, current methods of managing labs often involving manual monitoring and outdated systems are insufficient for providing comprehensive oversight. These limitations result in frequent system downtimes, poor resource allocation, and reactive rather than proactive maintenance practices. By developing this desktop application, the goal is to automate many of these tasks, using real-time data collection and predictive maintenance algorithms to provide lab managers with actionable insights. This approach is motivated by the growing importance of data-driven decision-making, where the ability to monitor system health and usage trends can significantly reduce inefficiencies, optimize resource utilization, and improve lab performance. Furthermore, reducing operational burdens on lab managers by automating resource tracking and system health monitoring will allow them to focus on higher-level administrative tasks, thereby improving the overall productivity of the lab. The motivation is to bridge the gap between current lab management challenges and the potential benefits of modern technological solutions.

1.3 Problem Definition and Objectives

Problem Definition:-

Develop a desktop application to track computer usage and system health for lab managers, with predictive maintenance capabilities to foresee system failures. The app will provide advanced reporting and analytics for resource optimization and decision-making. A user-friendly interface will ensure smooth lab operations.

Objectives:-

1. Monitor Computer Usage and System Health: Continuously track the usage patterns and health status of lab computers to ensure optimal functionality and performance.
2. Centralized Data Management: Collect access and system health data from all lab systems and send it to a primary server for centralized management and analysis.
3. Resource Optimization: Provide administrators with actionable insights into usage trends, helping them make informed decisions about resource allocation and improving overall lab efficiency.
4. Predictive Maintenance: Enable predictive maintenance capabilities, allowing administrators to anticipate system malfunctions or schedule repairs based on system health indicators and usage data.
5. Comprehensive Reporting and Analytics: Offer sophisticated visualization tools that provide detailed reports and analytics, supporting administrators in identifying patterns and making data-driven decisions.

1.4 Project Scope & Limitations

Project Scope:-

The Electron Eye Activity Tracking and Visualization Application will help lab managers monitor computer usage and system health across labs. It tracks user sessions, monitors performance metrics like CPU and memory, and sends data to a central server for analysis. With predictive maintenance, the system forecasts potential failures, minimizing downtime. Its advanced visualization and reporting tools provide insights for resource optimization, while a user-friendly interface ensures smooth lab operations and efficient management.

Limitations:-

The Electron Eye Activity Tracking and Visualization Application may face several limitations. One major concern is data privacy, as continuous monitoring of user sessions could raise privacy issues if not properly managed. Additionally, the application might encounter scalability challenges, especially as the number of systems increases, requiring more infrastructure to handle real-time data tracking and aggregation. Another limitation is its dependency on a centralized server, which means a server failure could disrupt data collection and analysis. Lastly, the initial setup and configuration could be complex and time-consuming, requiring significant technical expertise to ensure smooth implementation across lab environments.

1.5 Methodologies of Problem solving

The problem-solving methodology for this project follows a structured, iterative approach to tackle the inefficiencies in traditional lab management. The first step involves identifying the key problems faced by lab administrators, such as difficulties in monitoring system health in real-time, predicting system failures, and allocating resources effectively. Through consultations with stakeholders like lab managers and IT staff, these challenges are clearly defined and prioritized. A comprehensive analysis of existing lab management practices is conducted to pinpoint gaps in real-time monitoring, reporting, and predictive maintenance capabilities. This analysis forms the foundation for the system's design and

feature development. Once the problems are well understood, the team begins gathering requirements from stakeholders, focusing on features that will have the greatest impact on lab efficiency, such as real-time health monitoring, predictive failure alerts, and detailed usage reports.

After gathering requirements, the design and development phase follows, where the system's architecture is created with a focus on scalability and modularity. Each feature - whether it be real-time monitoring or predictive maintenance — is developed independently and integrated into the system. The development process is iterative, meaning that the system is continuously tested, refined, and improved based on feedback. For example, the predictive maintenance algorithm will undergo extensive testing to ensure that it can accurately predict hardware issues based on the collected data. Real-time monitoring will also be tested to ensure the system can handle large amounts of data without compromising performance. Once the core system is developed, it will be deployed in a controlled environment for beta testing. Lab administrators will use the system in real-world scenarios, and their feedback will be gathered to refine the system further. The iterative nature of this methodology ensures that the system is both reliable and adaptable to the specific needs of lab administrators, with ongoing updates planned based on usage feedback and system performance.

CHAPTER 2

LITERATURE SURVEY

The author[1] discuss a tool which is designed to enhance security and monitoring within an enterprise network by tracing and logging user activities on Linux-based systems. The tool uses various Linux commands, implemented in C programming, to monitor system parameters, maintain logs and store information in a database. It enables administrators to remotely access logs and monitor activities. The tool uses Mysql and PHP for efficient data retrieval and remote accessibility through a web interface. There are several areas where it can be improved and extended, such as support for multiple platforms and allow integration with other security tools.

This author[2] discusses building a test application known as the User Activity Tracker (UAT). The solution sought in this paper is to provide a quick and flexible way to track user activity on personal computers during forensic investigations. Some of the main challenges include finding and accessing log files, getting data from the registry, reading custom binary files, automating the timeline building process, and making reports that the user can understand. The UAT primarily obtains and processes timeline data from the registry, file system, event logs, internet history, and system restore points. All of this information can quickly be combined to demonstrate what a user was doing. This paper also includes a case study that demonstrates how the UAT tool works in comparison with some commercial forensics tools.

According to Author[3], CTTS is a module in a Project Management system that tracks employee activities. It allows employees to log start and end times for check-ins, task management, and generation of timesheet. The system improves upon task delegation and project performance monitoring from the manager's and staff members' perspectives. This system identifies three roles of the users: admin, manager, and employee. The admin role supervises employee and project details; the manager approves or rejects timesheets, and the employee is the one who records input from timesheet data and updates tasks. It is built on Struts 2 framework that develops with MVC architecture. Roll-out is phased, that is, the core module like login and task management will first appear. Amongst these benefits are better delegation, heightened monitoring of projects, and easier payroll processing.

The author[4] describes a log-driven prediction system which is created for complex IT systems and it focuses on predicting system failures using operational logs. The system monitors and analyzes the real-time logs generated by IT system to identify patterns that can indicate about the future failures. This system is based on automated extraction of features which removes the need of manual analysis and enables the model to quickly process large amount of data. The system uses Long Short-Term Memory (LSTM) for capturing long-term dependencies. This allows model to discover the long-range of historical data. Also the authors propose to explore online deep learning, which would enable the system to dynamically update the model as new failure events occur. It will improve the accuracy over the period of time.

In addition to existing tools, the author[5] suggests a new mobile application for monitoring crucial performance metrics in mobile apps, which is designed to overcome limitations such as obsolescence and lack of transparency. The suggested tool examines crucial data such as memory usage, CPU usage and charging history, battery life, traffic on computers and network status using Google's Android API. Additionally, this framework provides

flexibility for future enhancements. The tool is designed to help developers and general users identify performance issues and optimize mobile app usage across multiple apps. Its objective is to address the shortcomings of current monitoring solutions by providing a modern, flexible, and well-documented approach to monitoring mobile app performance. To improve app efficiency and user experience, the paper highlights the need to prioritize key operational aspects for real-time performance evaluation. Here are some potential areas for future scope:

- 1) Incorporate machine learning algorithms to predict performance degradation and resource exhaustion based on historical data trends.
- 2) Expand the tool to support monitoring on other mobile platforms, such as iOS, in addition to Android. This would make it a cross-platform solution for developers working on both Android and iOS apps, providing a unified monitoring experience.
- 3) Implement cloud integration to store monitoring data securely and perform advanced analytics on historical data.

According to the author[6], MARA is a methodology for monitoring applications and continuous performance assurance that employs Plan-Do-Check-Act cycles with four phases: Monitor, analyze data, recommend improvements, and take action. Two types of dashboards are introduced, namely Application Health Dashboards for real-time monitoring and Corrective Action Dashboards for addressing identified issues. The dashboards can be adjusted to different levels of detail, allowing IT teams to make informed decisions and respond quickly to problems. The method is applicable to industries like Telecommunication, Banking, E-commerce, and Healthcare, catering to mission-critical applications. The findings indicate that it aids in achieving business objectives, minimizes risks, and reduces costs by maintaining optimal performance. The MARA methodology can be enhanced by integrating AI and machine learning for predictive analytics, enabling proactive identification of potential performance issues. Expanding support for multi-cloud and hybrid environments would make it more versatile in diverse IT infrastructures.

This author[7] discusses a failure prediction method that uses Support Vector Machines (SVM) to predict failures which are based on information from log files. The method uses SVM, which is a machine learning algorithm known for its classification capabilities, to analyze log messages and identify patterns that may give rise to a system failure. This approach recognizes that log messages are sequential. The order and combination of log events can indicate about potential future issues. Log messages are represented using their tag values. The system uses a spectrum kernel to represent the sequence of log messages. While the method described can encounter limitations or lag in certain situations. As more features are added (such as message content, inter-arrival times, etc.), the number of features grows exponentially, increases the computational complexity of the model. This can make the system slower or less scalable when handling large datasets. The SVM-based approach may struggle with adapting to changing system behavior as it does not use online learning techniques. The system can be made more scalable, accurate, and adaptable to various machine types and environments. In summary, the collected research emphasizes the growing need for effective monitoring, logging, and prediction systems across various domains, such as enterprise networks, mobile applications, and complex IT infrastructures. Future enhancements can focus on incorporating online learning algorithms for adaptive monitoring, expanding cross-platform capabilities for unified monitoring experiences, and optimizing the scalability of prediction models for large datasets.

CHAPTER 3

**SOFTWARE REQUIREMENTS
SPECIFICATION**

3.1 Assumptions and Dependencies

Assumptions:-

1. Stable Network Connection: Users are expected to have reliable internet connectivity for optimal performance.
2. Modern Computer Systems: End-users will operate the application on systems capable of handling the software requirements.
3. User Familiarity with IT Concepts: Users are assumed to have basic knowledge of computer systems and monitoring tools.

Dependencies:-

1. Web Technologies: Reliance on web frameworks and databases for data management and server communication.
2. Third-Party Libraries: Integration with libraries for data visualization and predictive analytics may impact system performance.

3.2 Functional Requirements

3.2.1 System Feature 1: Real-Time System Monitoring

Description:-

The system must continuously monitor computer usage and health metrics across all connected machines.

Requirements:-

1. Users can view real-time data on CPU usage, memory consumption, and disk space.
2. Alerts must be generated for critical health metrics that fall below specified thresholds.
3. Historical data must be stored for trend analysis and reporting.

3.2.2 System Feature 2: Predictive Maintenance

Description:-

The platform will analyze usage data to predict when maintenance is required.

Requirements:-

1. The system must use machine learning algorithms to forecast potential failures based on historical data.
2. Maintenance schedules should be recommended to users based on predicted system health issues.
3. Users must receive notifications for upcoming maintenance tasks.

3.2.3 System Feature 3: Reporting and Analytics

Description:-

The application must provide detailed reports and analytics to aid administrators in decision-making.

Requirements:-

1. Users can generate reports summarizing system health and usage trends.

- 2. The reporting feature must allow for customization of report parameters (e.g., date range, specific metrics).
- 3. Reports must be exportable in various formats (e.g. PDF, Excel).

3.3 External Interface Requirements

3.3.1 User Interfaces

Feature	Description
Desktop Application Interface	A user-friendly interface designed for easy navigation and monitoring.
Reporting Dashboard	Visual representation of data analytics and system health metrics.

Table 3.1: User Interfaces

3.3.2 Hardware Interfaces

Hardware Component	Description
Computers	Compatible desktops or laptops for running the application.
Network Connection	Reliable internet connection for server communication and data reporting.

Table 3.2: Hardware Interfaces

3.3.3 Software Interfaces

Software Interface	Description
Data Visualization Library	Used for rendering real-time charts and graphs.
Database Management System	MongoDB for storing user data, system logs, and reports.

Table 3.3: Software Interfaces

3.4 Nonfunctional Requirements

3.4.1 Performance Requirements

1. Latency: The application should maintain latency below 100 milliseconds for real-time updates.
2. Concurrent Users: Must support up to 50 concurrent users without performance degradation.
3. Response Time: The application should respond to user actions within 1 second.

3.4.2 Safety Requirements & Security Requirements

1. Data Backup: Regular backups of user data and system logs to prevent data loss.
2. User Safety: Implement user reporting features for addressing safety concerns within the lab.
3. Data Encryption: All data exchanged between client and server must be encrypted.
4. User Authentication: Robust authentication mechanisms to ensure secure access to the application.

3.5 System Requirements

3.5.1 Database Requirements

1. Type: MongoDB database for efficient data management.
2. Capacity: Must support large datasets including user profiles, system logs, and historical data.

3.5.2 Software Requirements

1. Framework: Electron.js for desktop application development.
2. Languages: JavaScript and TypeScript for development.
3. Libraries: Chart.js for data visualization and Axios for HTTP requests.

3.5.3 Hardware Requirements

1. User Devices: Modern desktops or laptops with minimum specifications to run the application.
2. Internet: Stable high-speed internet connection for optimal performance.

3.6 Analysis Models: Agile Methodology to be applied

The Agile methodology will guide the development of the Electron Eye project, emphasizing iterative progress, flexibility, and continuous feedback through the following key practices:

1. Iterative Development: Divide the project into smaller increments for regular delivery of functional software.

2. Collaboration: Foster collaboration among team members and stakeholders to align with user needs continuously.
3. Adaptive Planning: Update plans and requirements based on real-time feedback.
4. Continuous Testing: Regular testing to identify and resolve issues early in the development cycle.
5. Customer Feedback: Incorporate user feedback to enhance the system iteratively.

This adaptable approach will facilitate rapid changes and continuous improvement throughout the project lifecycle.

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

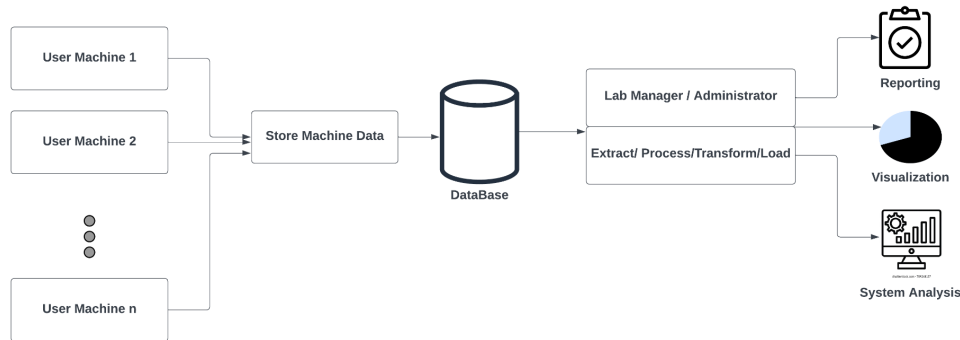


Figure 4.1: System Architecture

The System Architecture Diagram involves various components:-

1. User Machines (1, 2, ..., n):

Each user machine generates data related to system usage, performance, and activity. This data is sent to a central system for tracking and management purposes.

2. Store Machine Data:

The collected data from user machines is centralized and stored systematically. It includes details such as application usage, system health, and performance metrics.

3. Database:

A centralized database stores machine data for easy access and management. It supports future retrieval and analysis by organizing data from multiple machines.

4. Lab Manager / Administrator:

The lab manager monitors machine data to ensure optimal system performance. They use the system to analyze user behavior and machine health reports.

5. Extract / Process / Transform / Load (ETL):

Data is extracted from the database and processed for analysis. It undergoes transformation to prepare for reporting and visualization tasks.

6. Reporting, Visualization, and System Analysis:

Processed data is used to generate reports for system performance and user activities. Visualization and system analysis help in understanding patterns and optimizing the lab environment.

4.2 Mathematical Model

Let the system be represented as a tuple:

$$S = \{I, P, O, F, D, A, R\}$$

Where:

- I = Input Set
 $I = \{\text{Process_Details, Network_Details, Network_Requests, Connected_Devices, Browser_History, System_Health}\}$
- P = Set of Processes
 $P = \{\text{Python_Script, Data_Transmission, Backend_Processing, USB_Detection, RealTime_Alerting}\}$
- O = Output Set
 $O = \{\text{Visual_Analytics, Cheating_Alerts, System_Reports, Usage_Logs}\}$
- F = Functions
 $F = \{f_1, f_2, f_3, f_4, f_5\}$ where:
 - f_1 : Monitor system activities
 - f_2 : Detect unauthorized USB usage
 - f_3 : Transmit and store data securely
 - f_4 : Generate real-time alerts
 - f_5 : Visualize system data for admin
- D = Database
 $D = \{\text{MongoDB}\}$, where data is stored with MAC address as key.
- A = Actors
 $A = \{\text{Lab_PCs, Admin, Server}\}$
- R = Set of Rules/Conditions
 $R = \{\text{if unauthorized USB detected} \rightarrow \text{trigger alert,}$
 if high CPU usage $\rightarrow$ highlight in dashboard,
 if browser activity is flagged $\rightarrow$ log and notify admin}

4.3 Data Flow Diagrams

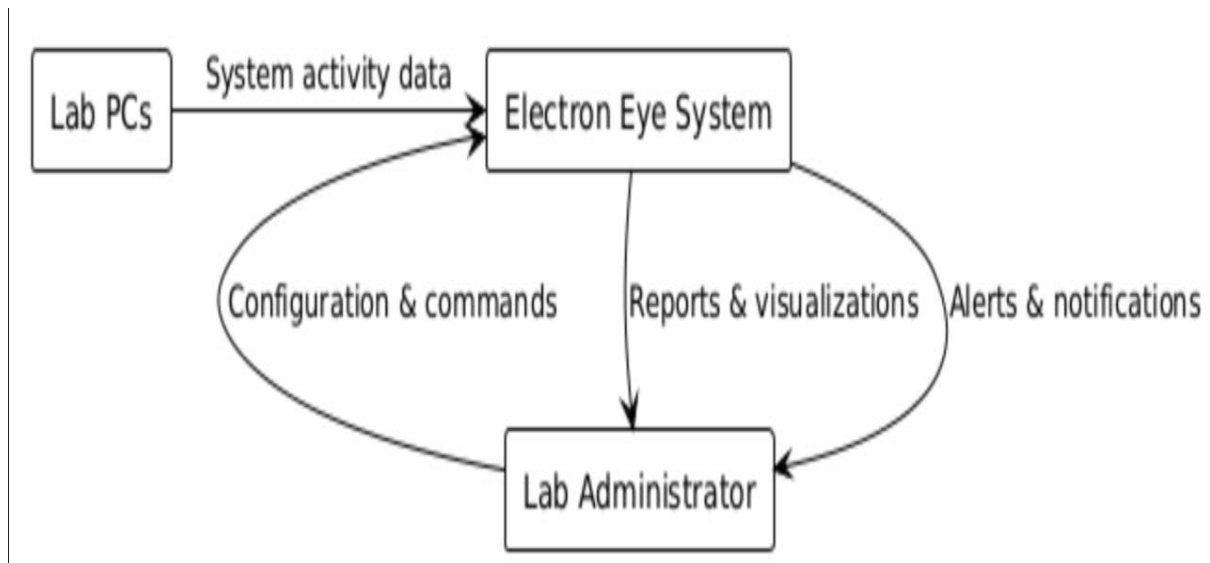


Figure 4.2: DFD Level 0 – Context Level

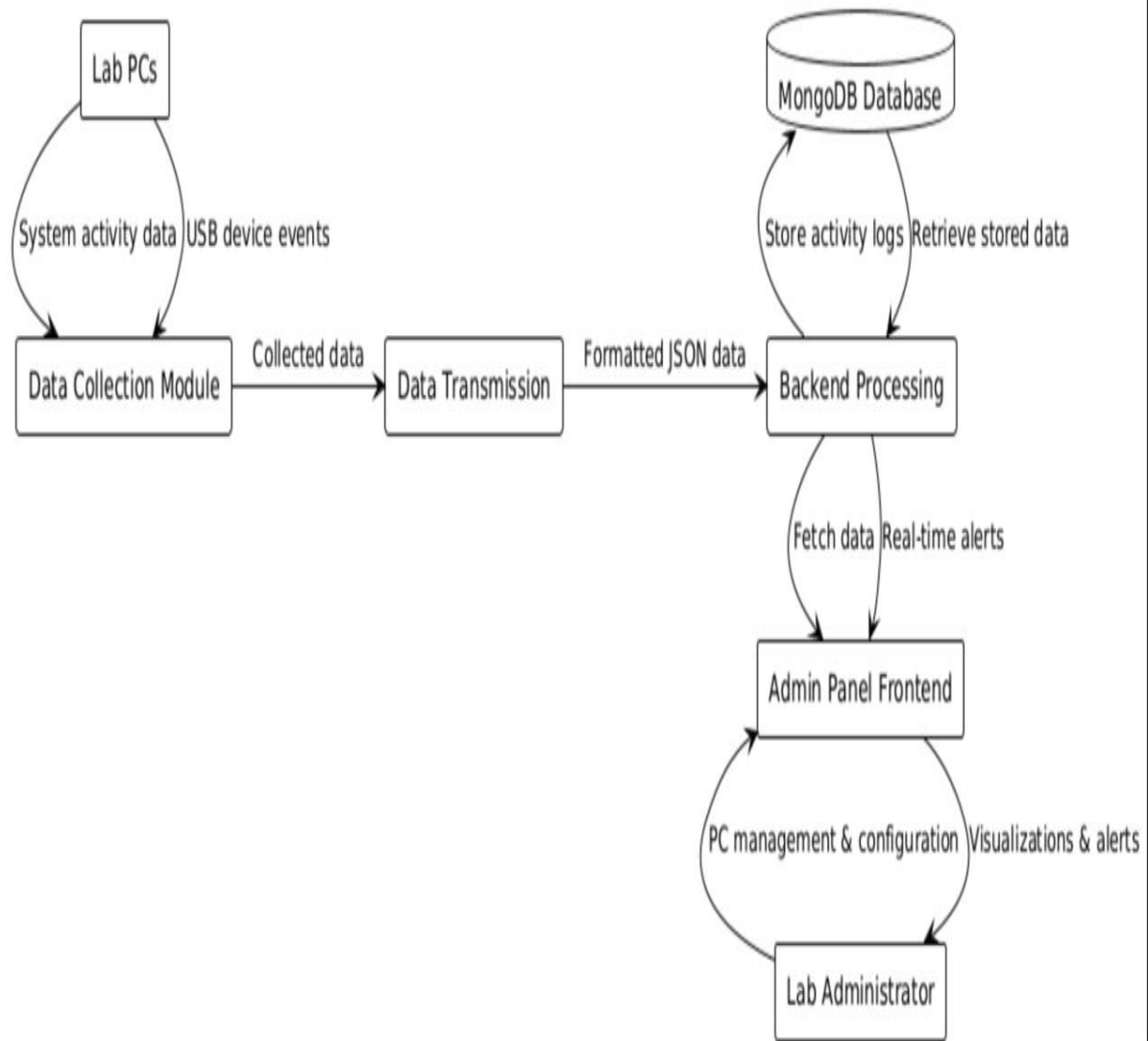


Figure 4.3: DFD Level 1 – Detailed View

4.4 Use Case Diagram

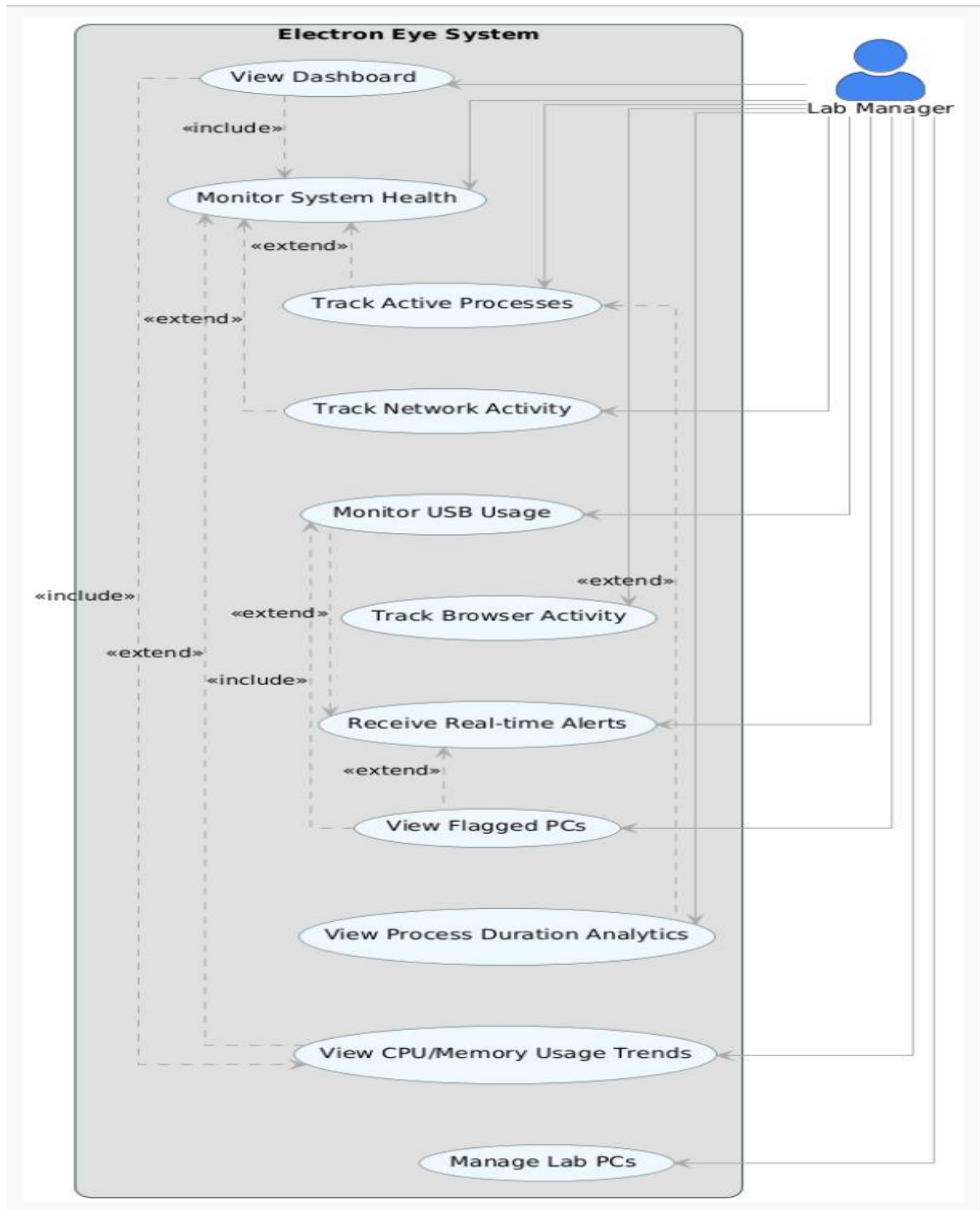


Figure 4.4: Use Case Diagram

4.5 Entity Relationship Diagram

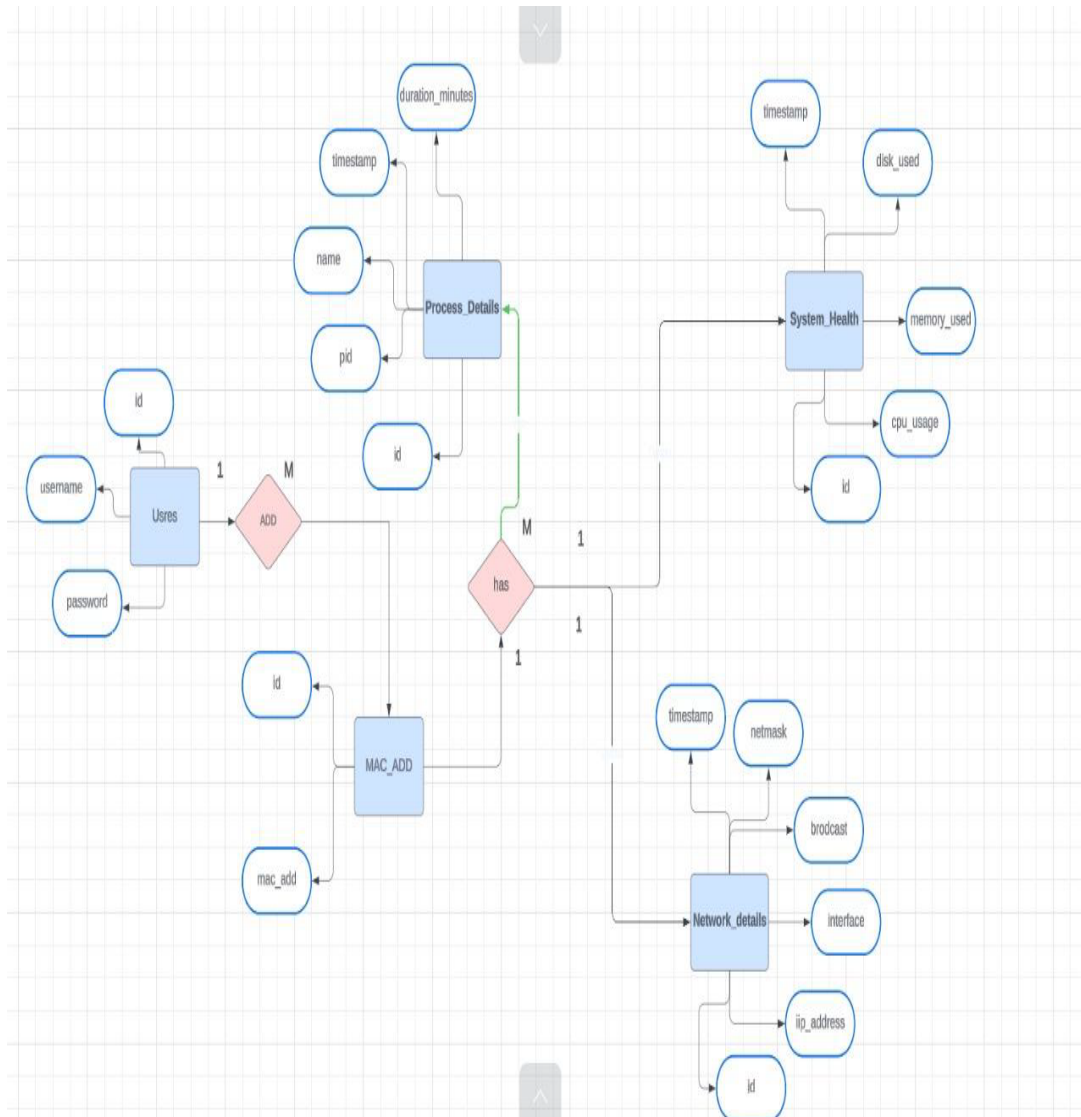


Figure 4.5: Entity Relationship Diagram

4.6 Class Diagram

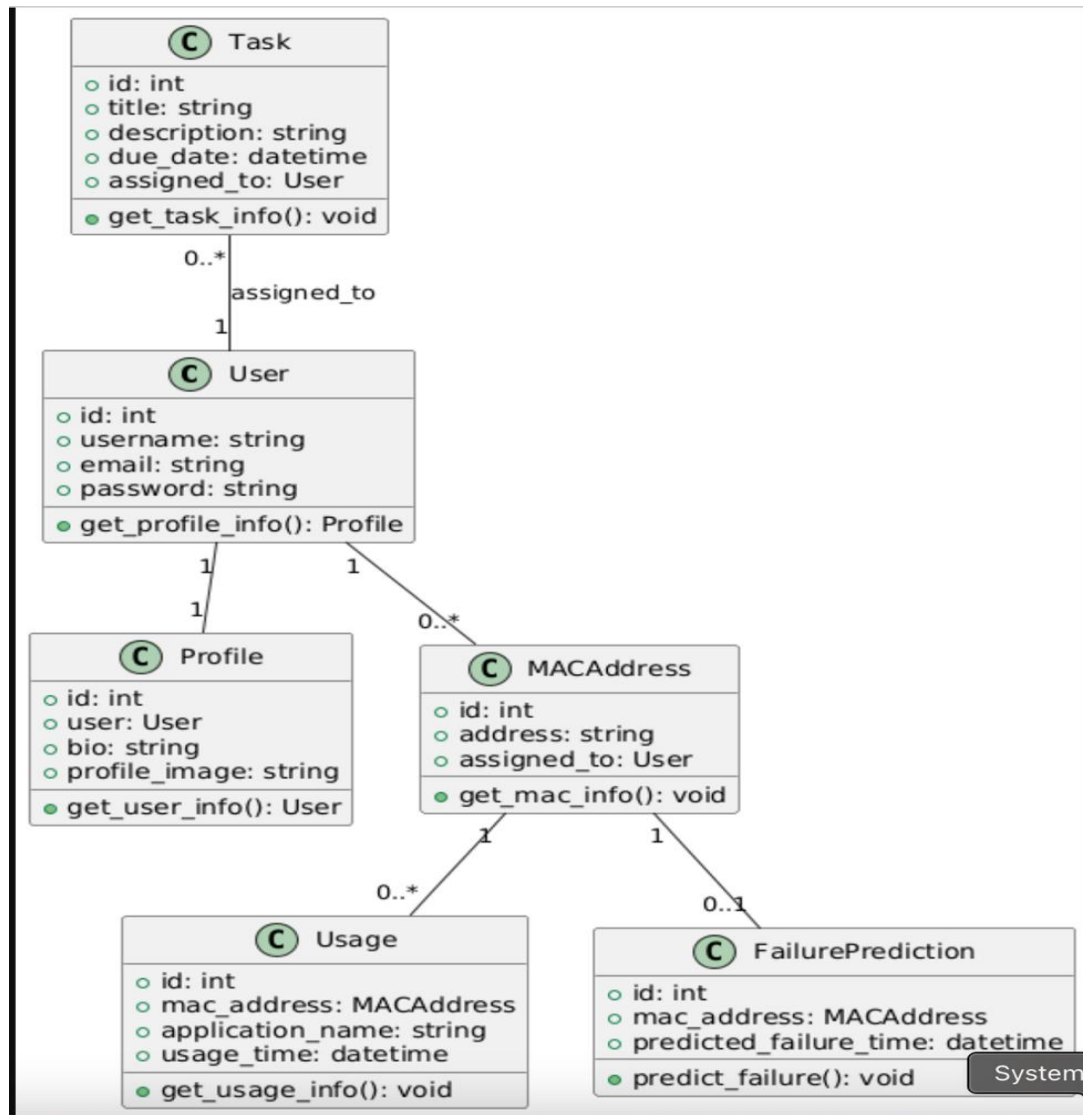


Figure 4.6: Class Diagram

4.7 Sequence Diagram

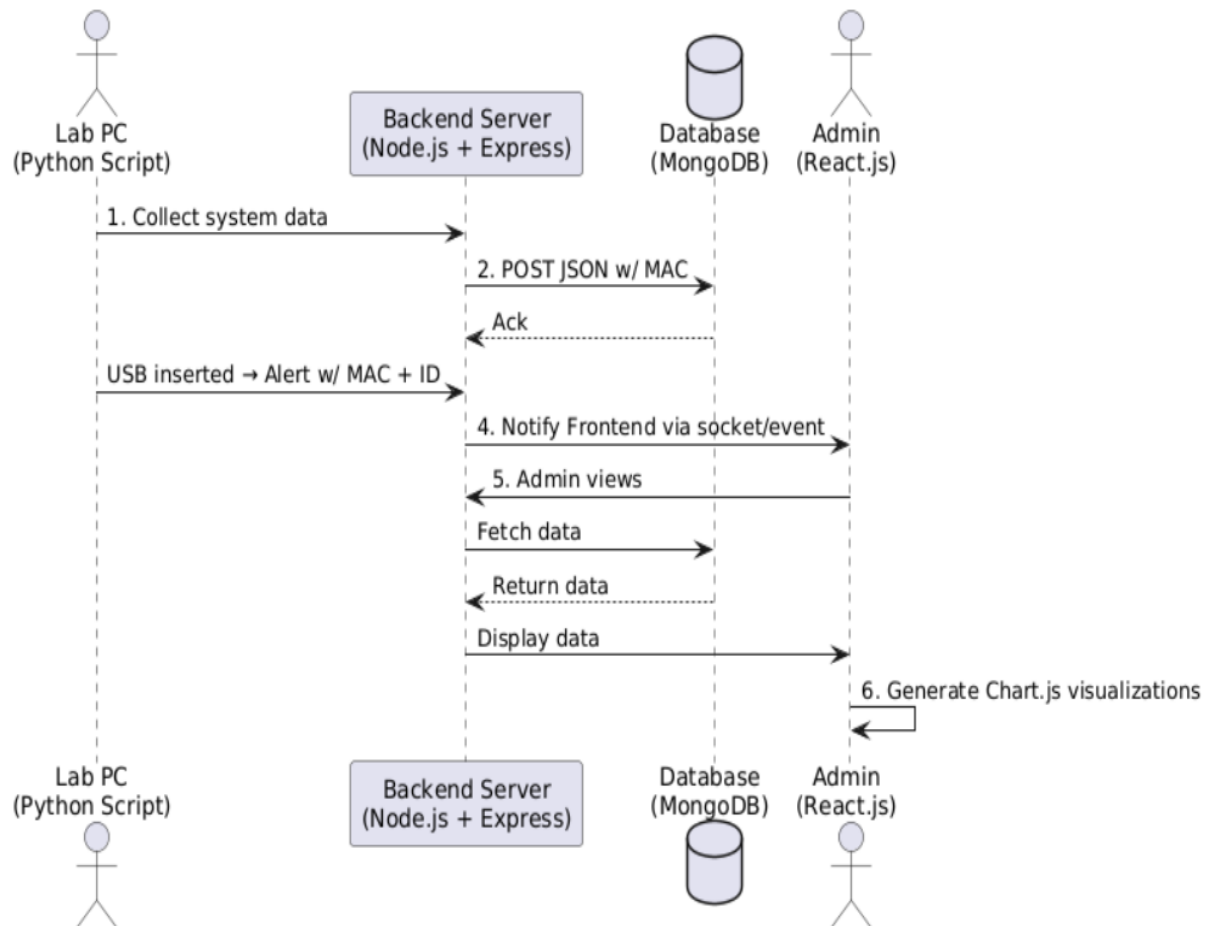


Figure 4.7: Sequence Diagram

4.8 Deployment Diagram

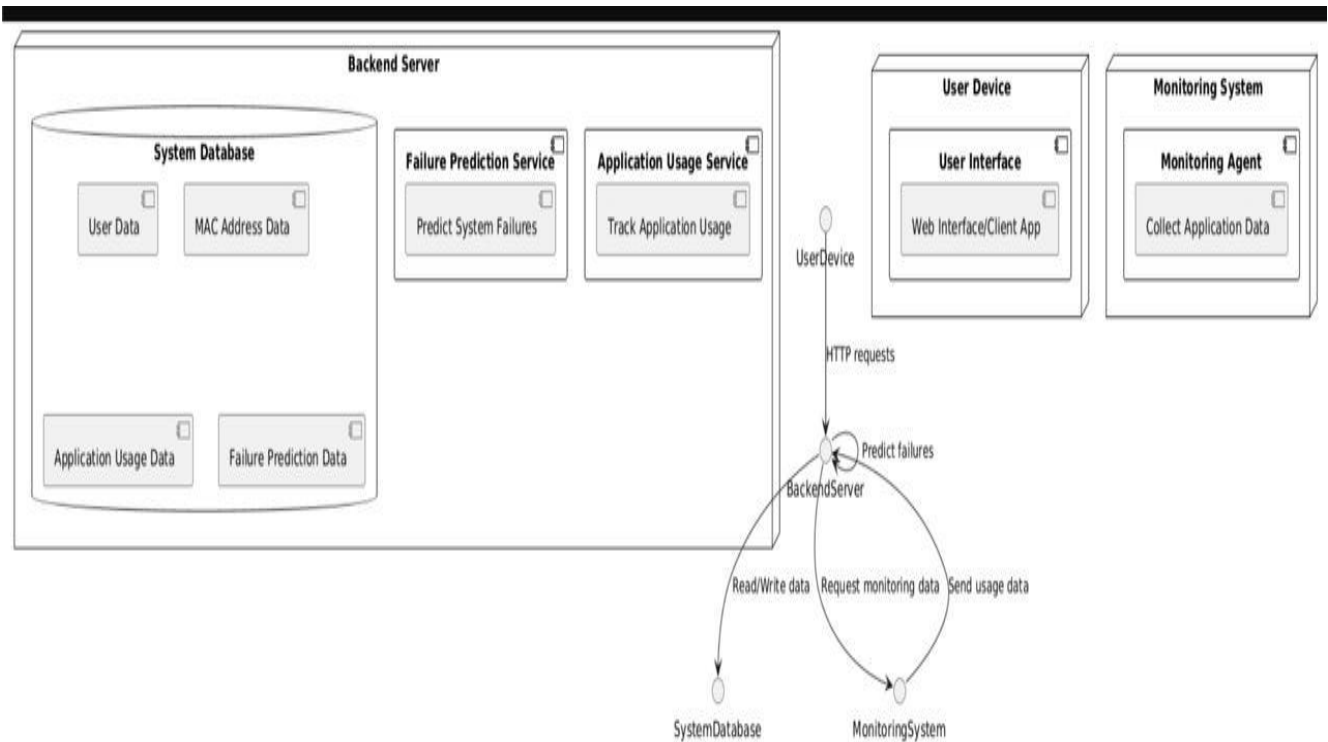


Figure 4.8: Deployment Diagram

CHAPTER 5

PROJECT PLAN

5.1 Project Estimate

5.1.1 Reconciled Estimates

1. Research and Development phase: Estimated at 6 months, focuses on building core features like usage tracking, predictive maintenance, and data visualization. It will provide insights for efficient lab management and resource allocation while ensuring a user-friendly interface to streamline operations and improve system health monitoring.
2. Testing and Evaluation: Estimated at 2 months, involves performance, security, and user acceptance testing. These tests will ensure efficient data handling, secure communication, and seamless usability for lab managers, validating the system's core functionalities.
3. Documentation and Review: Estimated at 1.5 months, will include creating user guides, technical documentation, and feedback-driven reviews. This ensures the system's usability, accuracy, and alignment with lab management needs.

5.1.2 Project Resources

1. Frontend Developer

- Responsible for designing and developing the user interface using HTML, CSS, JavaScript, and React.js.
- Create a user-friendly and intuitive interface to streamline lab management tasks.
- Implement responsive design to ensure compatibility across various devices and screen sizes.

2. Backend Developer

- Develop and manage the backend infrastructure using Node.js and Python.
- Handle data processing, server-side logic, and integration with the frontend.
- Implement security measures for data handling and communication with the database.

3. Database Technology

- Utilize MongoDB for efficient data storage and retrieval, enabling flexible schema design.
- Ensure proper data modeling to support usage tracking and system health monitoring.
- Implement aggregation queries to analyze data and generate insights for administrators.

4. Collaboration Tools

- Use GitHub for version control, enabling collaboration and tracking code changes.
- Implement Google Meet for team communication and real-time collaboration during development.
- Utilize project management tools (e.g., Trello or Jira) for task assignment and progress tracking.

5.2 Risk Management

Risk management is essential for the successful implementation of the lab management desktop application. By identifying potential risks early, we can mitigate their impact on development and deployment.

1. **Technical Risks:** Integration challenges between technologies like React.js and MongoDB may arise. To mitigate this, thorough testing will be conducted at each development phase.
2. **Data Security Risks:** Handling sensitive usage data poses security threats. We will implement robust security measures, including encryption and secure access controls, to protect against breaches.
3. **User Acceptance Risks:** The application's success relies on its usability for lab managers. User feedback will be gathered during testing to refine the interface and features.
4. **Project Timeline Risks:** Delays due to unforeseen challenges could impact the timeline. We will adopt an agile approach with regular progress reviews to address issues promptly.

By proactively managing these risks, we aim to enhance the project's reliability and ensure a smooth deployment of the application.

5.2.1 Risk Identification

Risk identification is crucial for anticipating potential issues that could impact the lab management desktop application's development and deployment. The following risks have been identified:

- **Technical Integration Risks:** Difficulty in integrating different technologies (React.js for the frontend, Node.js and MongoDB for the backend) could lead to functionality issues.
- **Data Security Risks:** Potential breaches or unauthorized access to sensitive usage data may compromise data integrity and confidentiality.
- **User Experience Risks:** The application may not meet the usability needs of lab managers, leading to low user adoption or dissatisfaction.
- **Performance Risks:** High data loads or inefficient queries could result in slow system performance, affecting user experience and system responsiveness.
- **Resource Risks:** Limited availability of skilled developers or unforeseen team changes may delay project timelines and impact quality.
- **Compliance Risks:** Failing to adhere to regulatory requirements related to data privacy and security could lead to legal issues and fines.
- **Change Management Risks:** Resistance to adopting the new system from lab staff could hinder successful implementation and usage.

5.2.2 Risk Analysis

Risk analysis involves evaluating the identified risks to determine their potential impact on the lab management desktop application project and the likelihood of their occurrence. This process helps prioritize risks and inform mitigation strategies.

- **Technical Integration Risks:**

- **Impact:** High – If integration issues occur, it can halt development and delay the project.
- **Likelihood:** Medium – While integration challenges are common, careful planning and testing can reduce the risk.

- **Data Security Risks:**

- **Impact:** Critical – A data breach can lead to significant legal and reputational consequences.
- **Likelihood:** Medium – With proper security measures, the risk can be minimized, but vulnerabilities may still exist.

- **User Experience Risks:**

- **Impact:** Moderate – If the application is not user-friendly, it may result in low adoption and decreased productivity.
- **Likelihood:** Medium – User feedback during development can mitigate this risk, but misalignment with user needs may still occur.

- **Performance Risks:**

- **Impact:** High – Poor performance can lead to frustration and hinder lab operations.
- **Likelihood:** Medium – The risk can be managed through optimization and performance testing.

- **Resource Risks:**

- **Impact:** Moderate – Limited resources may cause delays in development and affect project quality.
- **Likelihood:** High – Staffing changes or resource limitations are common in projects, making this a significant risk.

- **Compliance Risks:**

- **Impact:** Critical – Non-compliance with data privacy regulations can result in legal action and fines.
- **Likelihood:** Low – With adequate legal oversight, compliance issues can be minimized.

- **Change Management Risks:**

- **Impact:** Moderate – Resistance to change may slow down the implementation process and reduce system effectiveness.

- **Likelihood:** Medium – Proactive training and communication can alleviate resistance, but some pushback is likely.

5.3 Project Schedule

5.3.1 Project Task Set

1. **Conduct a Needs Assessment:** Survey lab managers to understand their challenges in tracking computer usage and system health.
2. **Meet with Stakeholders:** Arrange meetings with lab administrators and relevant stakeholders to gather requirements and expectations for the lab management application.
3. **Formal Discussions:** Engage in discussions with college authorities to align project goals with institutional needs and expectations.
4. **Brainstorming Session:** Collaborate with the development team to brainstorm solutions based on the gathered requirements.
5. **Documentation:** Maintain proper documentation outlining the problem statement, requirements, and stakeholder concerns.
6. **Technology Stack Identification:** Evaluate and select the technology stack, including frontend and backend technologies, based on feasibility and project requirements.
7. **Research Innovative Solutions:** Investigate advanced technologies and methodologies to address prevalent issues in lab management and system monitoring.
8. **Team Assignments:** Assign specific modules and tasks to team members based on their expertise and experience.
9. **Learning Phase:** Begin learning the required technologies, including React.js, Node.js, and MongoDB, necessary for developing the application.
10. **Design User Interfaces:** Create interactive and user-friendly interface designs for the application.
11. **Design Approval:** Finalize the user interface designs and obtain approval from stakeholders before development begins.
12. **Development Phase:** Develop user interfaces and functional modules concurrently to streamline the development process.
13. **Model Identification:** Identify suitable machine learning models for predictive maintenance and usage analysis.
14. **Model Development and Testing:** Develop and rigorously test the chosen machine learning model to ensure accuracy and reliability.
15. **Integration Phase:** Integrate the frontend, backend, and machine learning model to create a cohesive application.

16. **Deployment:** Deploy the application and assess its functionality and performance based on feedback from lab managers and stakeholders.
17. **Iterative Improvement:** Continuously seek advancements and enhancements that can be made within the project based on user feedback and technological developments.
18. **User Testing and Feedback:** Conduct testing sessions with lab managers to gather reviews and assess the effectiveness of the application.
19. **Documentation:** Document the entire development process, including codebase, user guides, and system requirements.
20. **Project Submission:** Submit the final project and accompanying documentation for future reference and maintenance.

5.3.2 Timeline Chart

This section represents a summarized project timeline chart. It briefly explains the schedule of our entire project from inception to projected completion.

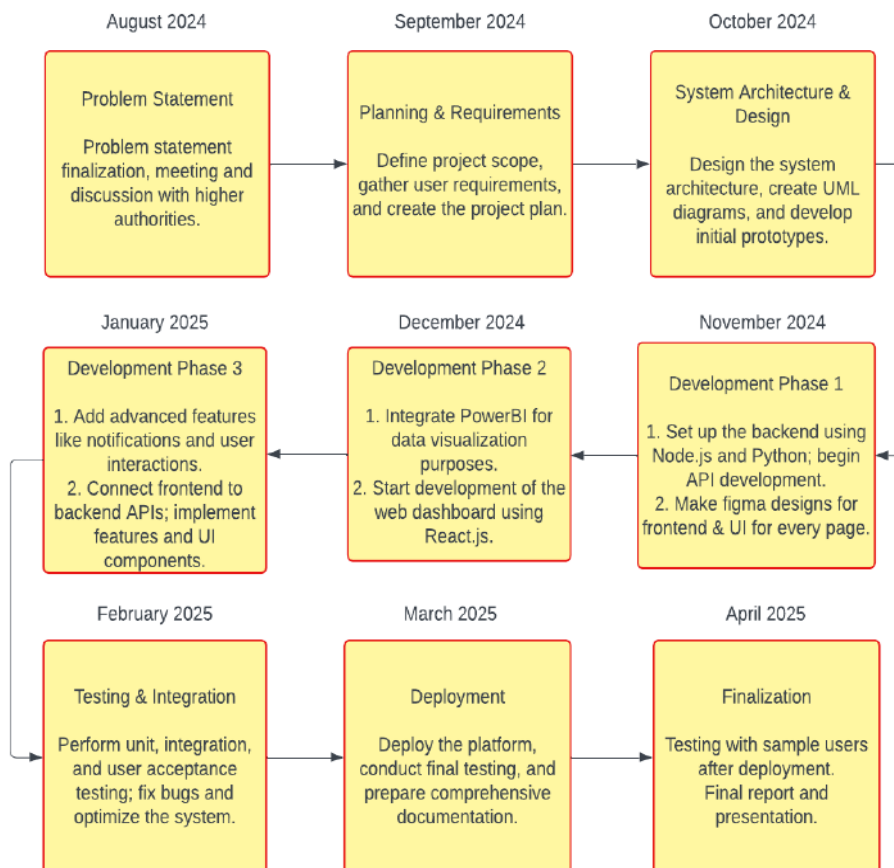


Figure 5.1: Summarized TimeLine Chart

5.4 Team Organization

5.4.1 Team structure

- **Rutwik Malav:** Backend Developer, UI/UX Designer.
- **Shruti Maliye:** Frontend Developer, System Designer.
- **Manav Mehta:** Database Engineer, Backend Developer.
- **Nikita More:** Frontend Developer, Documentarian.
- **Dr. K. C. Waghmare:** Internal Mentor and Guide.
- **Dr. P. T. Kulkarni (Director, PICT):** Advisor and Higher Leadership.

5.4.2 Management reporting and communication

Management

For management during our project, we used several effective tools and practices to ensure regular and timely updates. We used **Google Groups** for team communication, where we regularly shared updates, research papers, and important documents. Additionally, we collected various research papers related to our project and organized as well as structured them properly within **Google Drive**. This made it easy for the entire team to access and refer to the collected material. For code management and version control, we used **Git** and **GitHub**, ensuring that all team members could collaborate, track changes, and contribute to the project.

Reporting

Bi-weekly, we reported our progress to the internal mentor, and provided updates on work completed, challenges faced, and ideas we came up with. The regular communication allowed us to stay aligned with project goals and deadlines of submissions. We consulted our advisor **Dr. P. T. Kulkarni** for new ideas and strategic guidance, which we will be implementing in our project, ensuring improvements and innovation.

Communication

For regular communication with team members, we used **Google Groups** and **WhatsApp** for quick updates, making it easy to share new information and get immediate feedback. Our communication with the mentor was structured around bi-weekly sessions and regular updates, with tasks given accordingly.

Thus, this structured approach to management, reporting, and communication helped us to maintain clarity, accountability, and efficiency throughout the project.

CHAPTER 6

PROJECT IMPLEMENTATION

6.1 Overview of Project Modules

The Electron Eye system comprises the following core modules:

- **Data Collection Module:** A Python script continuously runs on each lab PC, collecting information such as process usage, network details, connected USB devices, browser history, and system health metrics.
- **USB Detection and Alert Module:** Monitors for unauthorized USB usage and sends real-time alerts to the admin panel with device information and timestamps.
- **Data Transmission Module:** Formats collected data as JSON and securely transmits it via HTTP POST to the backend server with MAC address tagging.
- **Backend Processing Module:** Handles data validation, storage, and API management. Built using Node.js and Express.js with MongoDB as the database.
- **Notification Module:** Triggers immediate alerts for suspicious activity, ensuring real-time responsiveness in the admin dashboard.
- **Admin Panel (Frontend) Module:** A React.js interface that allows lab managers to view per-PC activity, flagged devices, and system performance through intuitive dashboards.
- **Visualization Module:** Uses Chart.js to display interactive graphs and time-based system analytics for efficient lab oversight.

6.2 Tools and Technologies Used

The following tools and technologies were employed to develop the Electron Eye system:

- **Frontend:** React.js – For building a responsive and interactive admin panel.
- **Backend:** Node.js and Express.js – For managing APIs, processing data, and communicating with the database.
- **Database:** MongoDB – Used for efficient and schema-flexible data storage, indexed using MAC addresses.
- **Scripting Language:** Python – For monitoring scripts running on lab PCs that collect system data and send updates.
- **Visualization:** Chart.js – For creating interactive data visualizations and monitoring dashboards.
- **Security:** Encryption techniques and secure communication protocols – To ensure the integrity and confidentiality of transmitted data.
- **Version Control:** GitHub – For source code management and collaboration.
- **Communication and Coordination:** Google Meet, Trello – For team meetings, task tracking, and agile project management.

6.3 Algorithm Details

6.3.1 Algorithm 1: Data Collection Algorithm

- Continuously run a Python monitoring script on each lab PC.
- Collect process details, network activity, connected devices, browser history, and system health metrics.
- Structure data into a standardized JSON format.

6.3.2 Algorithm 2: USB Detection and Alert Algorithm

- Detect new USB device insertions using platform-specific libraries (e.g., pyudev for Linux or win32file for Windows).
- On detection, extract device ID, MAC address, and timestamp.
- Immediately trigger an alert and send the event to the backend server.

6.3.3 Algorithm 3: Real-time Notification and Dashboard Update

- Backend server receives and validates incoming alerts or activity data.
- If the activity is flagged (e.g., USB insert, unauthorized app), it pushes an alert to the admin dashboard.
- The frontend interface shows real-time pop-ups and logs triggered by server-side events.

6.3.4 Algorithm 4: Data Transmission to Backend

- Collect structured data from the monitoring script.
- Append the MAC address of the current lab PC to ensure unique identification.
- Send data via HTTP POST to the backend server at regular intervals.
- Implement encryption protocols for secure data transmission.

6.3.5 Algorithm 5: Backend Data Storage and Management

- Receive data packets from lab PCs.
- Validate incoming JSON structure and sanitize the data.
- Store the data into MongoDB, indexed by MAC address and timestamp.
- Update or append system logs based on the data type (e.g., process, network, USB, browser).

6.3.6 Algorithm 6: Flagged Device Identification

- Monitor incoming data for rule violations (e.g., USB insertions, restricted browser activity).
- If a violation is detected, mark the corresponding MAC address as “flagged”.
- Log event details (timestamp, type, MAC) and notify the admin.

6.3.7 Algorithm 7: Visualization Rendering Algorithm

- On dashboard load or filter update, request data from backend APIs.
- Use Chart.js to dynamically generate visual representations such as:
 - CPU/RAM usage over time (line chart)
 - Active processes and their duration (bar chart)
 - USB activity logs (table)
- Provide filtering options by date, MAC address, and activity type.

6.3.8 Algorithm 8: Admin Panel PC Monitoring View

- Display a list of all registered lab PCs with their MAC address and status.
- On selecting a PC, query the backend for its activity logs.
- Render a detailed page showing all historical and live monitoring data:
 - Processes
 - Network requests
 - Connected devices
 - Browser history
 - System health indicators

CHAPTER 7

SOFTWARE TESTING

7.1 Types of Testing

The following types of testing were conducted to ensure the robustness and reliability of the Electron Eye desktop application:

- **Unit Testing:** Individual modules such as USB detection, data transmission, and backend processing were tested independently to verify their correctness and behavior in isolation.
- **Integration Testing:** The interaction between frontend and backend components (React.js and Node.js APIs), as well as communication between Python scripts and the server, were tested for seamless data flow and system coordination.
- **System Testing:** The entire application was tested end-to-end, covering the data collection from lab PCs, real-time alert generation, and visualization in the admin panel.
- **Functional Testing:** All core functionalities like activity logging, USB detection, and visual analytics were validated against functional requirements to ensure they work as intended.
- **Security Testing:** Emphasis was placed on verifying that sensitive data is transmitted securely and that access controls are enforced at both the backend and frontend.
- **Performance Testing:** The system's responsiveness and behavior under load (high data volume from multiple PCs) were evaluated to confirm that performance remains optimal.
- **User Acceptance Testing (UAT):** Conducted with lab managers to gather feedback on usability, interface layout, and effectiveness in real lab scenarios.

7.2 Test cases & Test Results

Below are representative test cases along with their expected and actual outcomes during the testing phase:

- **Test Case 1: USB Detection**
 - **Input:** Insert a USB device into a monitored lab PC.
 - **Expected Output:** Real-time alert sent to the backend and displayed on admin dashboard.
 - **Actual Result:** Alert successfully received and displayed. **(Pass)**
- **Test Case 2: Unauthorized Browser Activity Logging**
 - **Input:** Open restricted browsers like Firefox or Edge.
 - **Expected Output:** Browser activity logged and device flagged in the admin panel.

- **Actual Result:** Activity captured and flagged correctly. **(Pass)**
- **Test Case 3: System Health Monitoring**
 - **Input:** Run multiple heavy applications to increase CPU usage.
 - **Expected Output:** Spikes in CPU usage shown in real-time graph.
 - **Actual Result:** Line graph accurately displayed performance metrics. **(Pass)**
- **Test Case 4: Backend Data Validation**
 - **Input:** Send malformed JSON data from the client.
 - **Expected Output:** Backend rejects request and returns error.
 - **Actual Result:** Error correctly returned and data not stored. **(Pass)**
- **Test Case 5: Admin Dashboard Filtering**
 - **Input:** Filter logs by a specific MAC address and date range.
 - **Expected Output:** Display relevant records only.
 - **Actual Result:** Logs filtered and displayed correctly. **(Pass)**

CHAPTER 8

RESULTS

8.1 Outcomes

- Lab administrators may remotely monitor and examine system activities like process use, USB connections, and browsing history thanks to the system's reliable and real-time monitoring solution for lab environments.
- Through a real-time notification system coupled with the admin dashboard, the application provides immediate alerts for unauthorized activity, facilitating prompt administrative action.
- Managers may make informed decisions about lab resource management and maintenance scheduling by using the interactive data visualization enabled by Chart.js to glean insightful information from both historical and current data.

8.2 Screen Shots

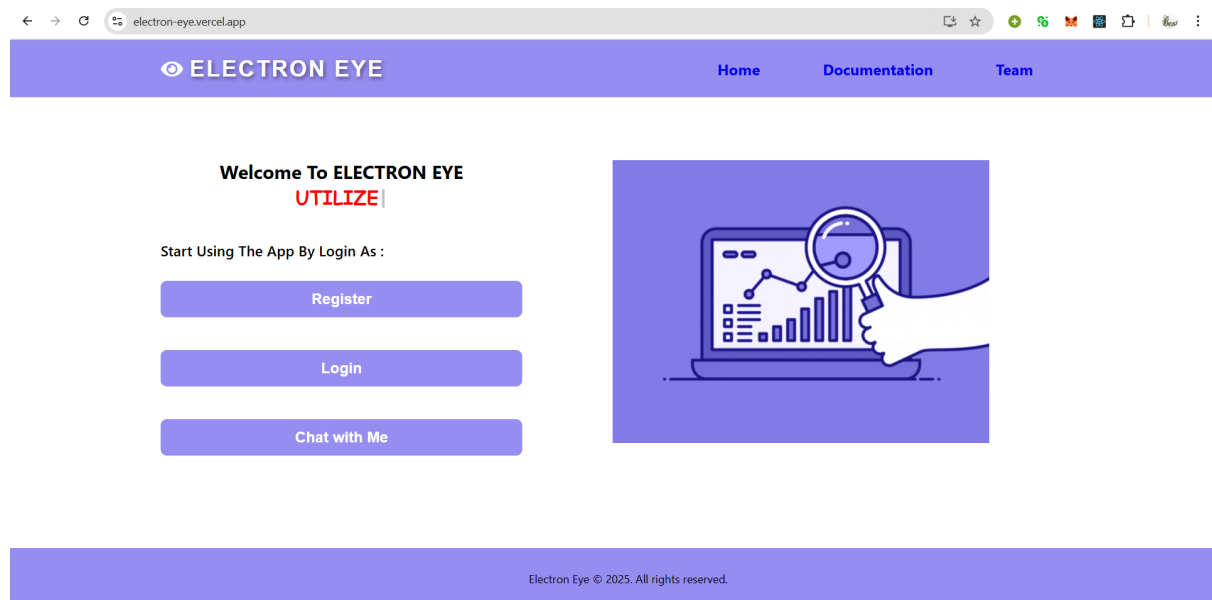


Figure 8.1: Home Page

The screenshot shows a web browser at the URL `electron-eye.vercel.app/Register`. The page has a purple header with the "ELECTRON EYE" logo and navigation links for "Home", "Documentation", and "Team". The main content area is a white box titled "Register User" containing three input fields: "Username:", "Password:", and "Lab ID:". Below these fields is a purple "Register" button. A link "Already have an account? Log in" is positioned below the button. The footer is a purple bar with the text "Electron Eye © 2025. All rights reserved."

Figure 8.2: Register Page

The screenshot shows a web browser at the URL `electron-eye.vercel.app/Login`. The page has a purple header with the "ELECTRON EYE" logo and navigation links for "Home", "Documentation", and "Team". The main content area is a white box titled "Login User" containing two input fields: "Username:" (with the text "manav" entered) and "Password:" (with masked characters "....." entered). Below these fields is a purple "Submit" button. A link "Not registered? Create an Account" is positioned below the button. The footer is a purple bar with the text "Electron Eye © 2025. All rights reserved."

Figure 8.3: Login Page

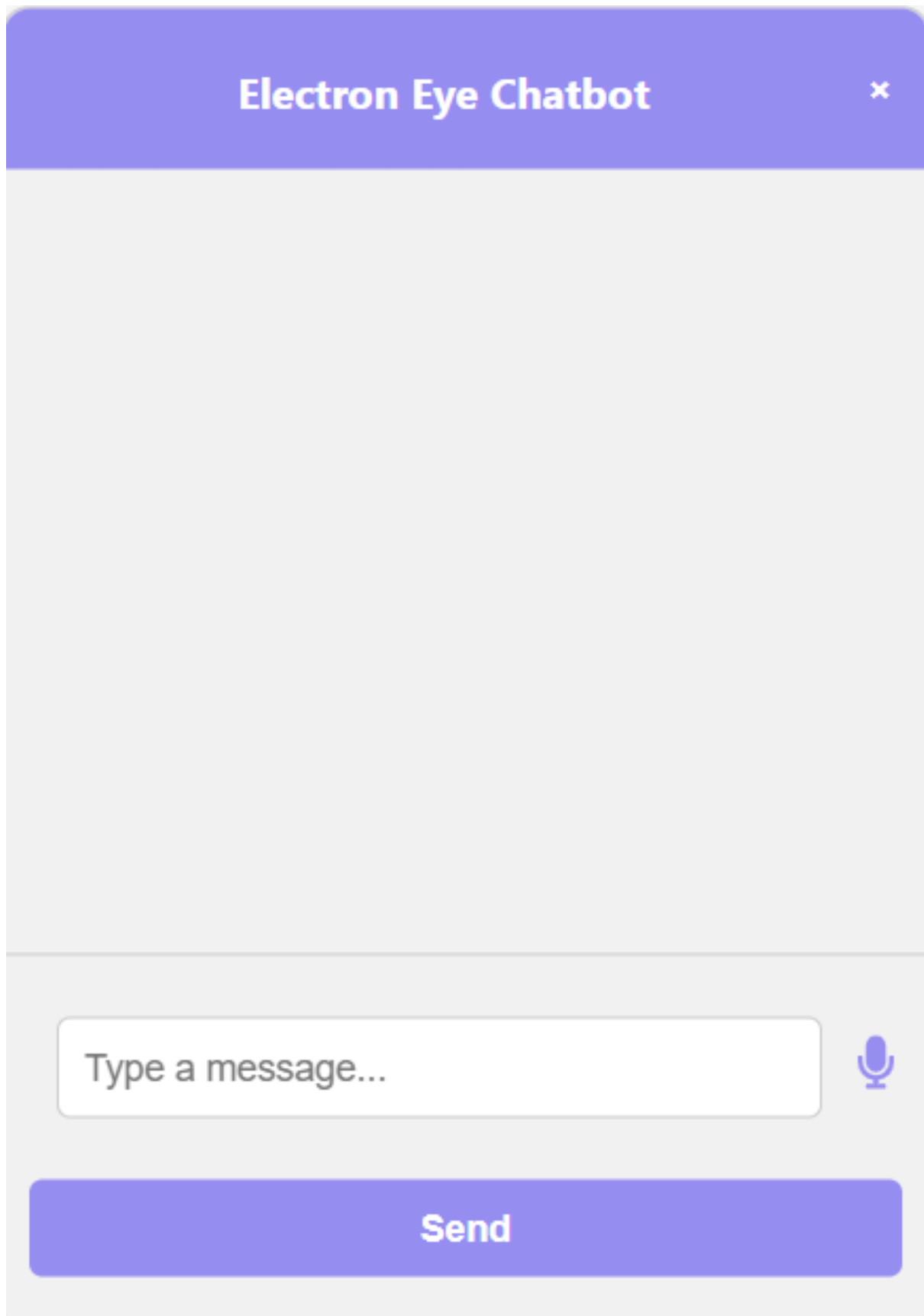


Figure 8.4: Chatbot

Add MAC Address

Add

MAC Address List

MAC Address	Machine Name
CC-47-40-2F-54-E8	M1
F4:8E:38:87:41:18	M3
E2-12-95-DB-57-AF	M2

Display Cheating Devices

View Graph of Cheating Devices

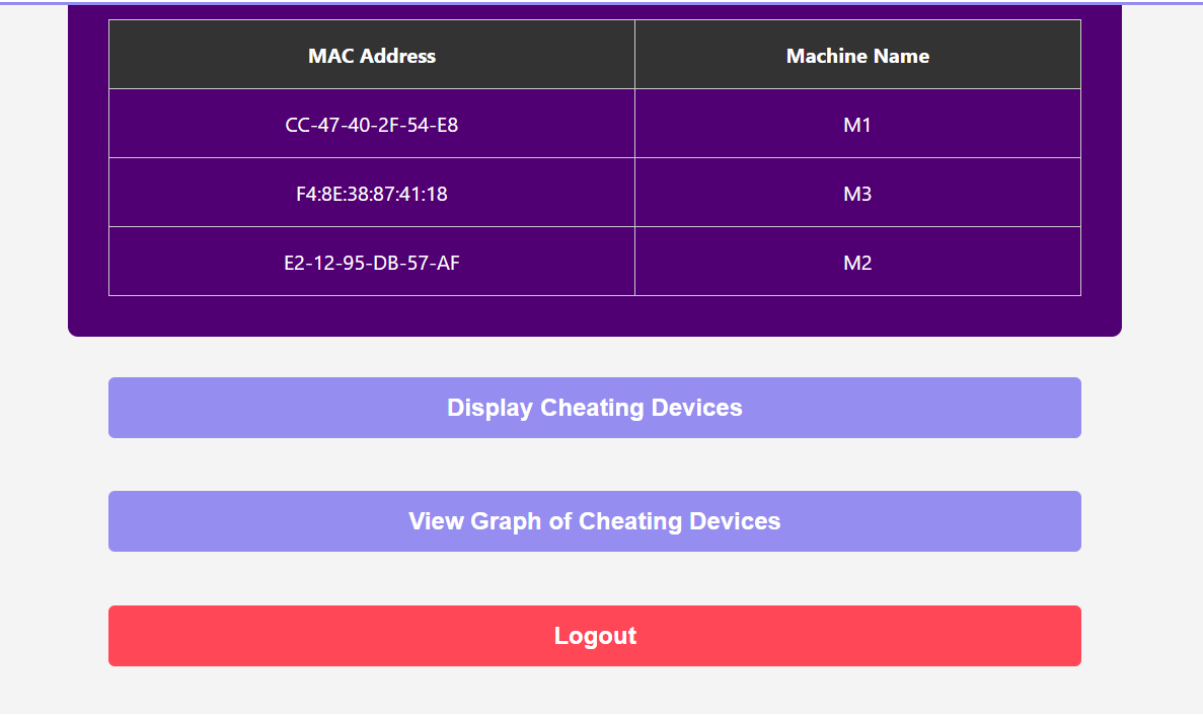


Figure 8.5: User interface for managing MAC Addresses, including adding new entries, viewing stored devices, and accessing further device analysis options.

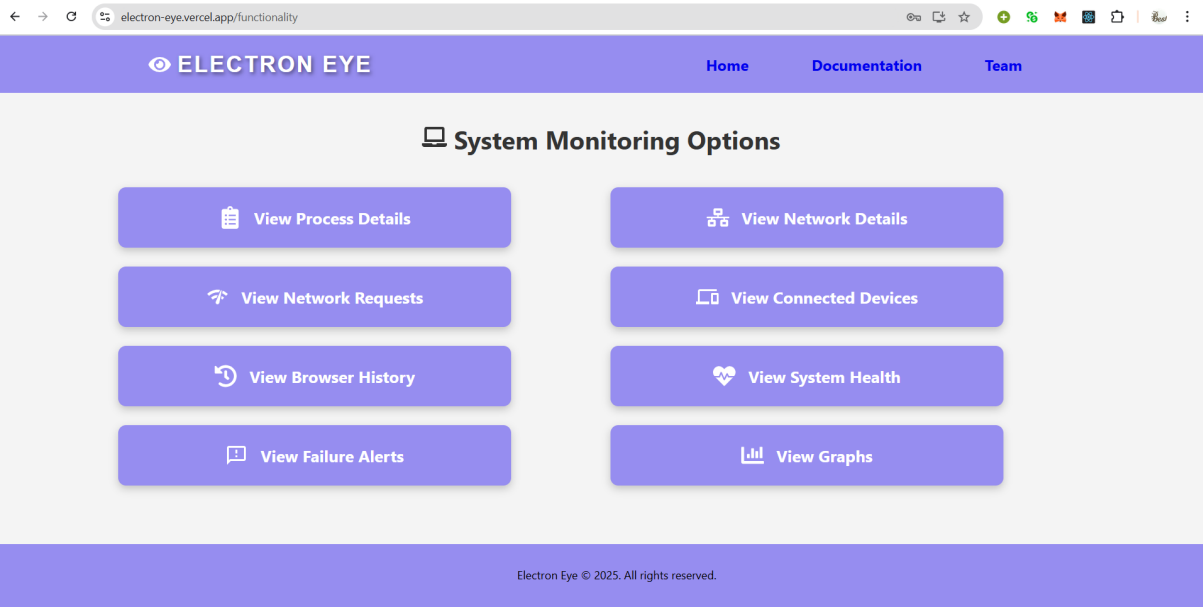


Figure 8.6: Various Features for a particular MAC Address

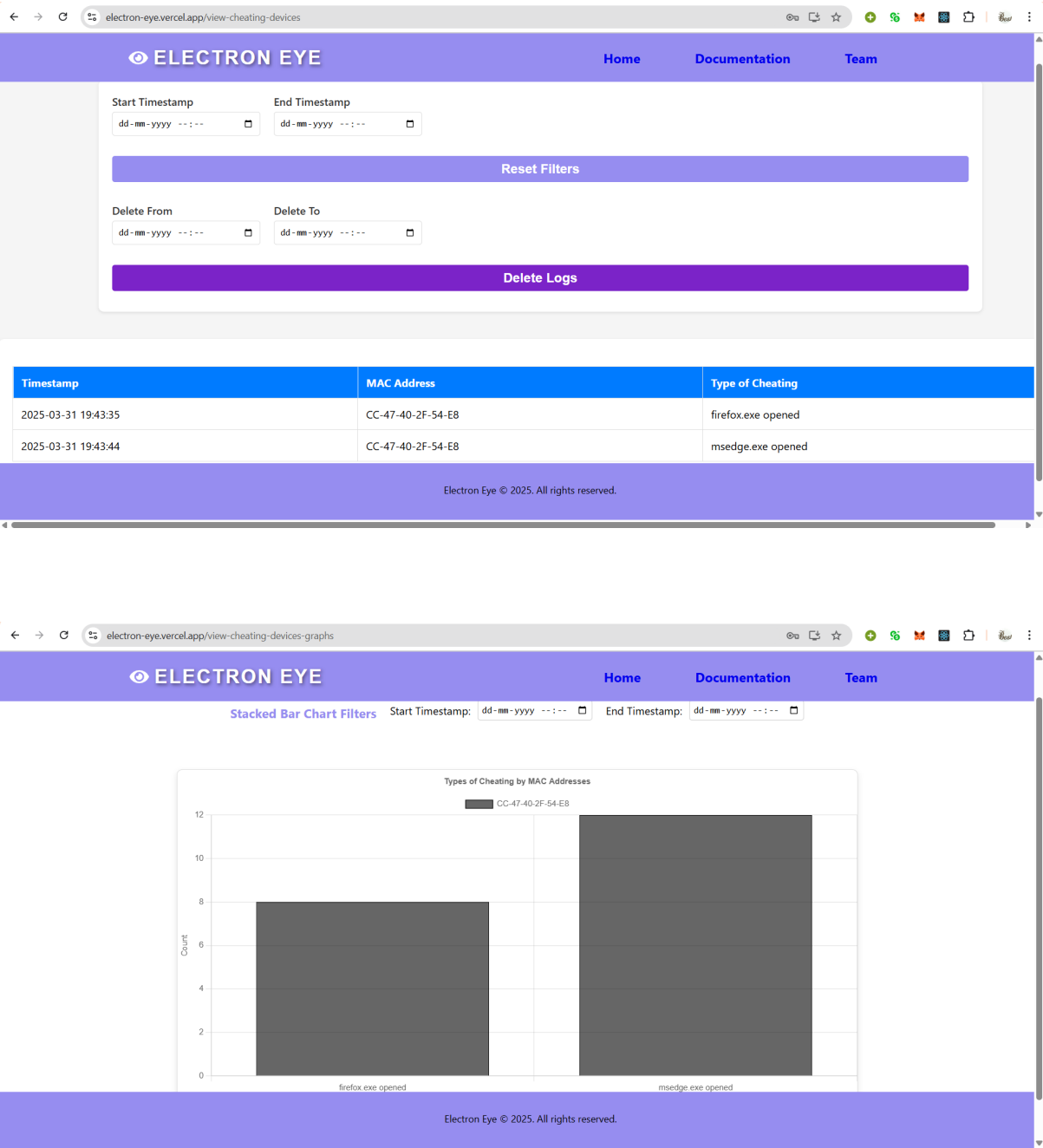


Figure 8.7: Cheating Devices

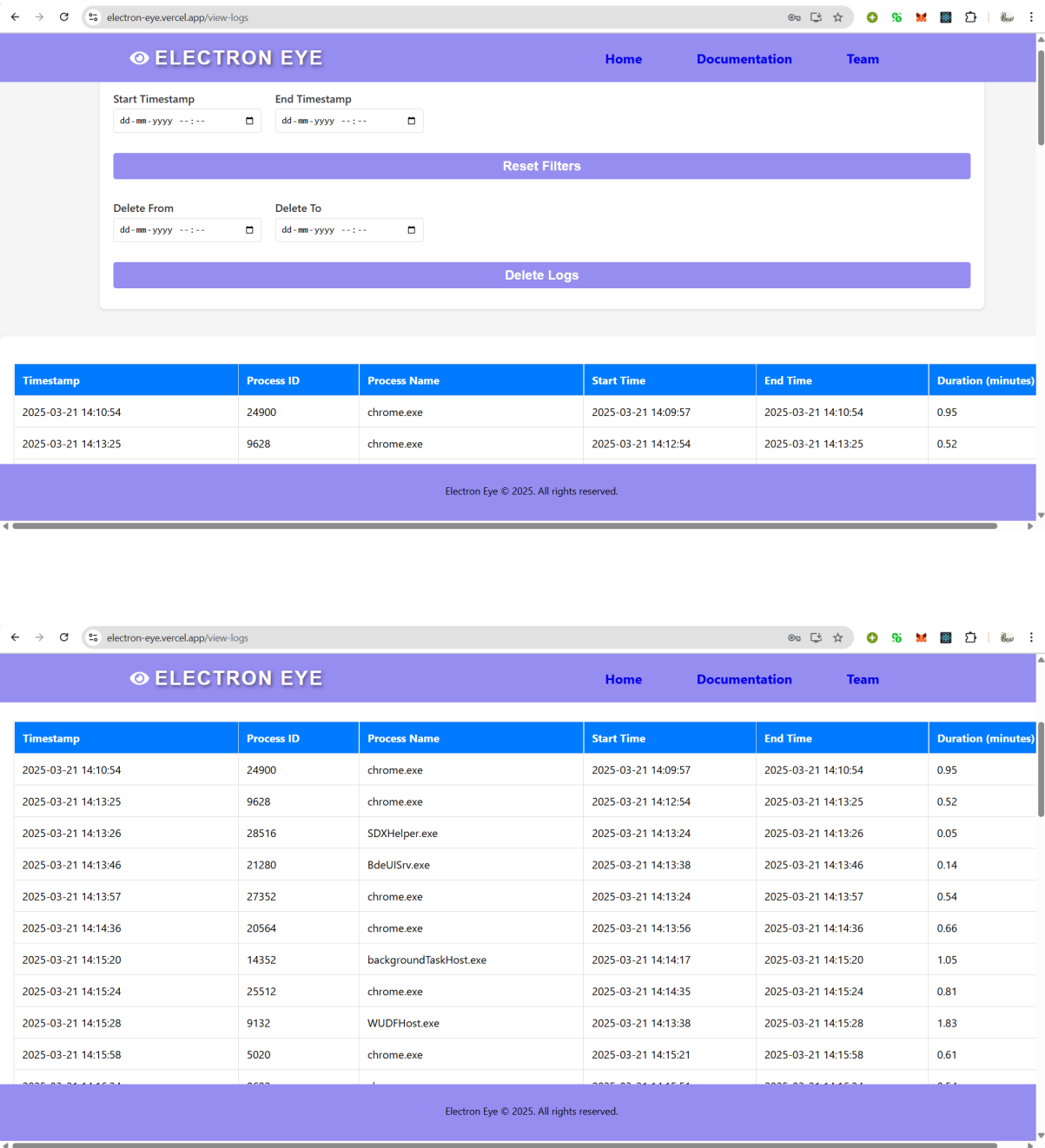


Figure 8.8: Process Details

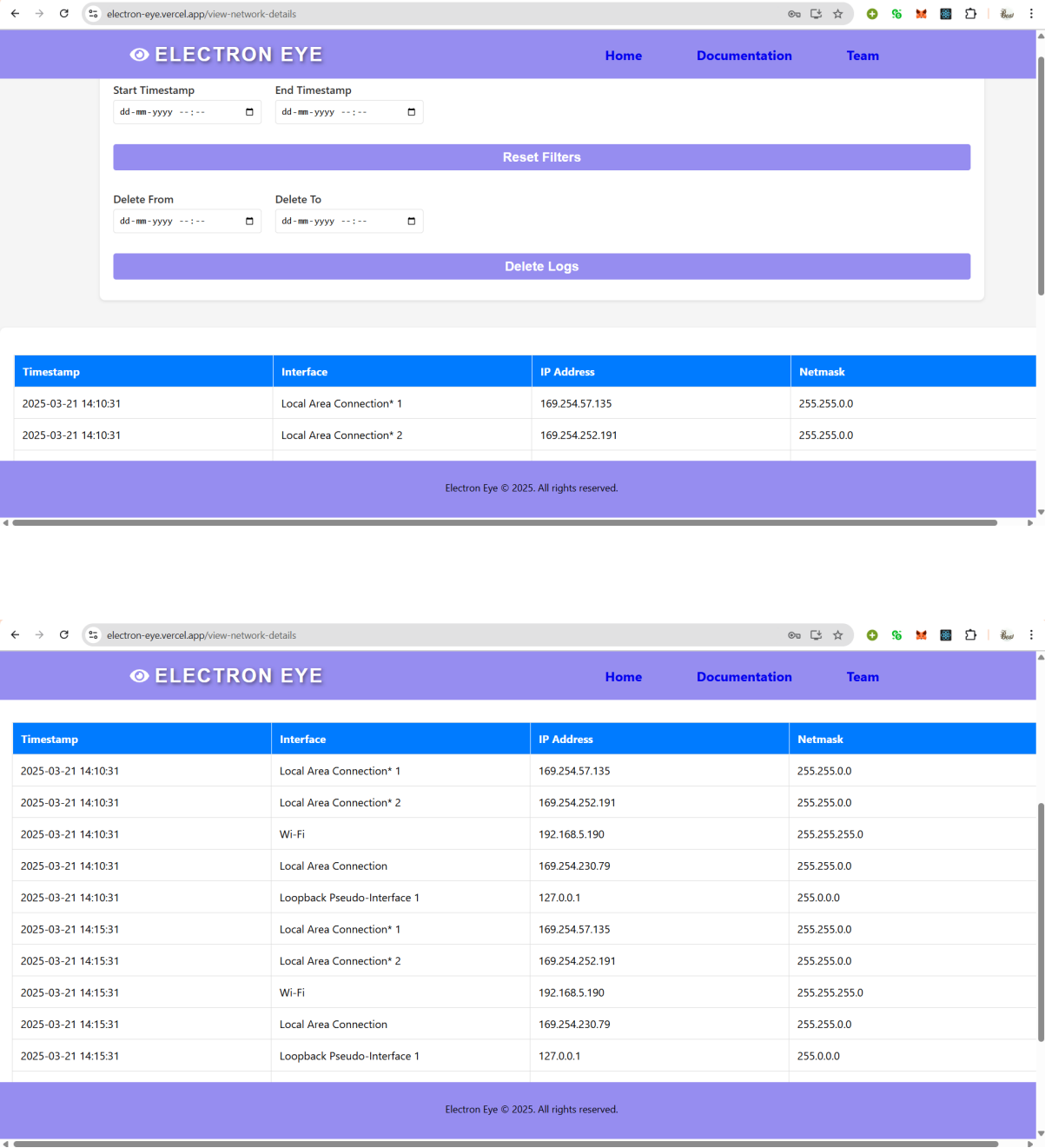


Figure 8.9: Network Details

Electron Eye: Activity Tracking and Visualisation Application

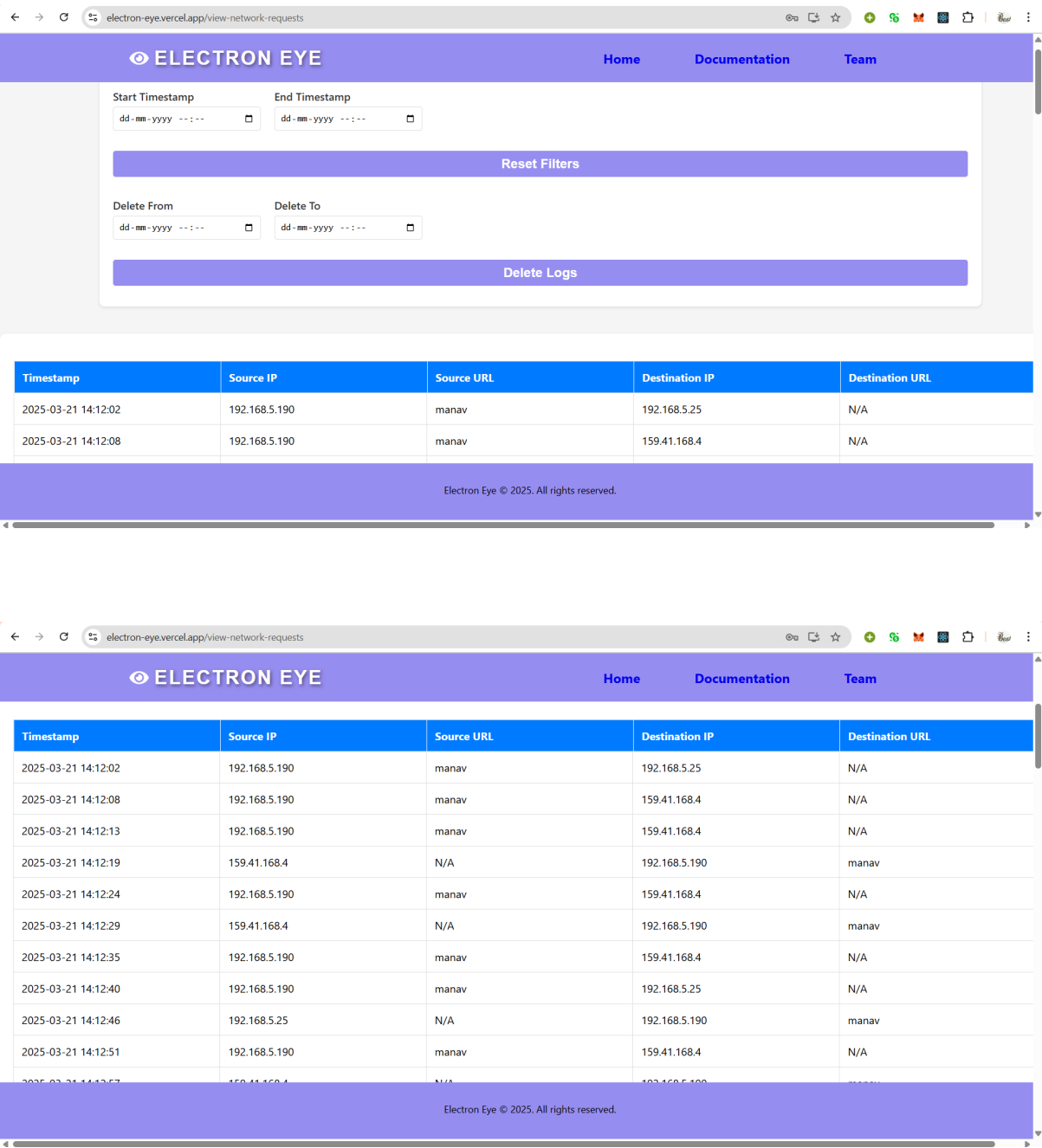


Figure 8.10: Network Requests

Electron Eye: Activity Tracking and Visualisation Application

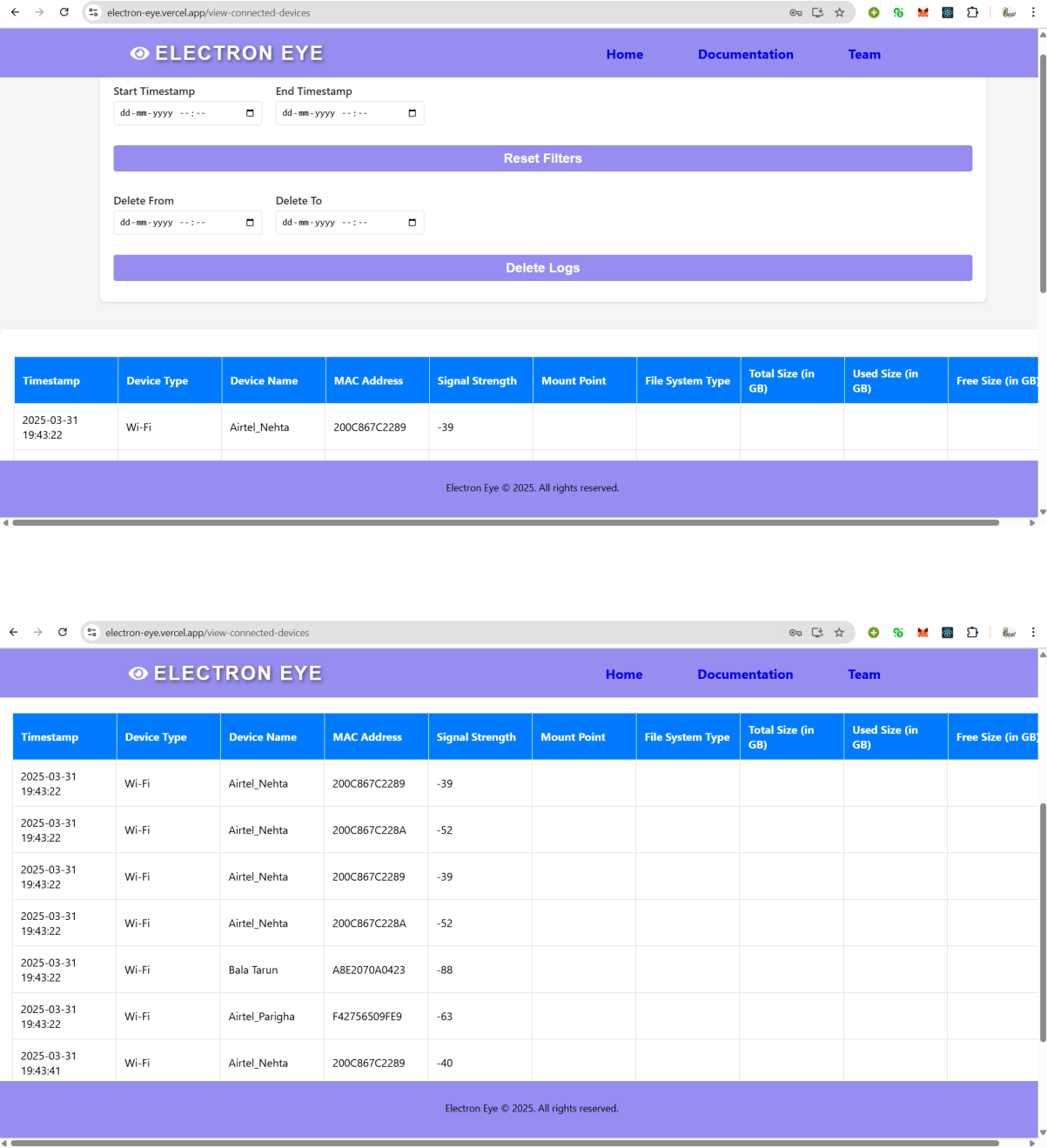
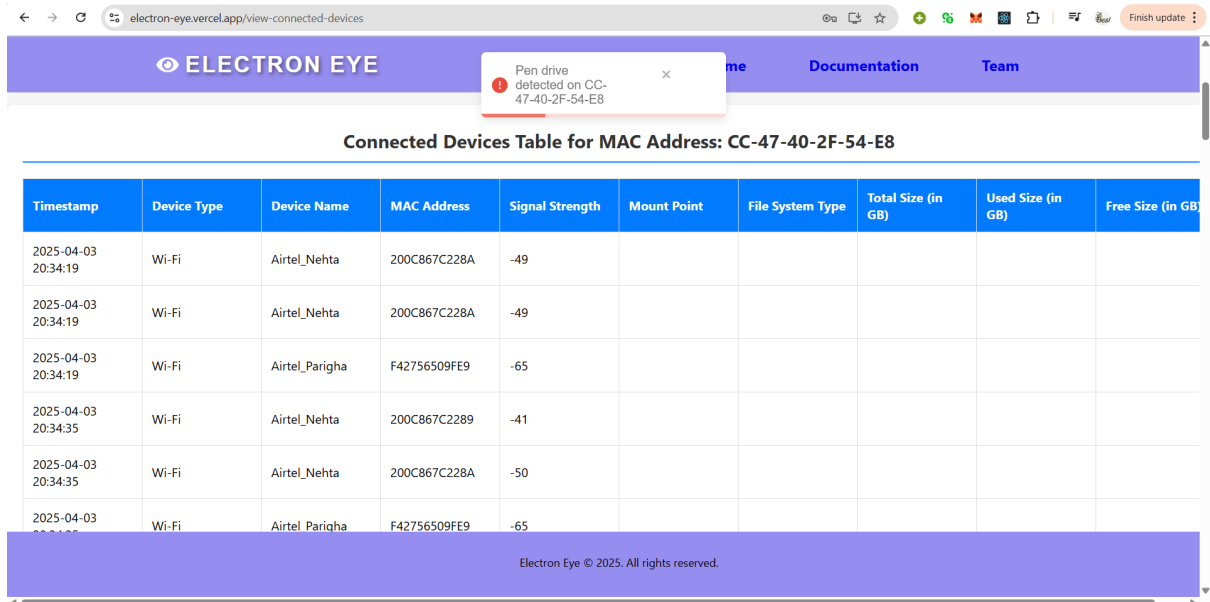


Figure 8.11: Connected Devices



The screenshot displays the Electron Eye application interface. At the top, a purple header bar contains the 'ELECTRON EYE' logo and navigation links for 'Home', 'Documentation', and 'Team'. A notification alert is visible, stating 'Pen drive detected on CC-47-40-2F-54-E8'. Below the header, a section titled 'Connected Devices Table for MAC Address: CC-47-40-2F-54-E8' features a table with the following data:

Timestamp	Device Type	Device Name	MAC Address	Signal Strength	Mount Point	File System Type	Total Size (in GB)	Used Size (in GB)	Free Size (in GB)
2025-04-03 20:34:19	Wi-Fi	Airtel_Nehta	200C867C228A	-49					
2025-04-03 20:34:19	Wi-Fi	Airtel_Nehta	200C867C228A	-49					
2025-04-03 20:34:19	Wi-Fi	Airtel_Parigha	F42756509FE9	-65					
2025-04-03 20:34:35	Wi-Fi	Airtel_Nehta	200C867C2289	-41					
2025-04-03 20:34:35	Wi-Fi	Airtel_Nehta	200C867C228A	-50					
2025-04-03 20:34:35	Wi-Fi	Airtel_Parigha	F42756509FE9	-65					

The footer of the application displays the copyright notice: 'Electron Eye © 2025. All rights reserved.'

Figure 8.12: Alert when Pen drive is detected on a particular MAC Address

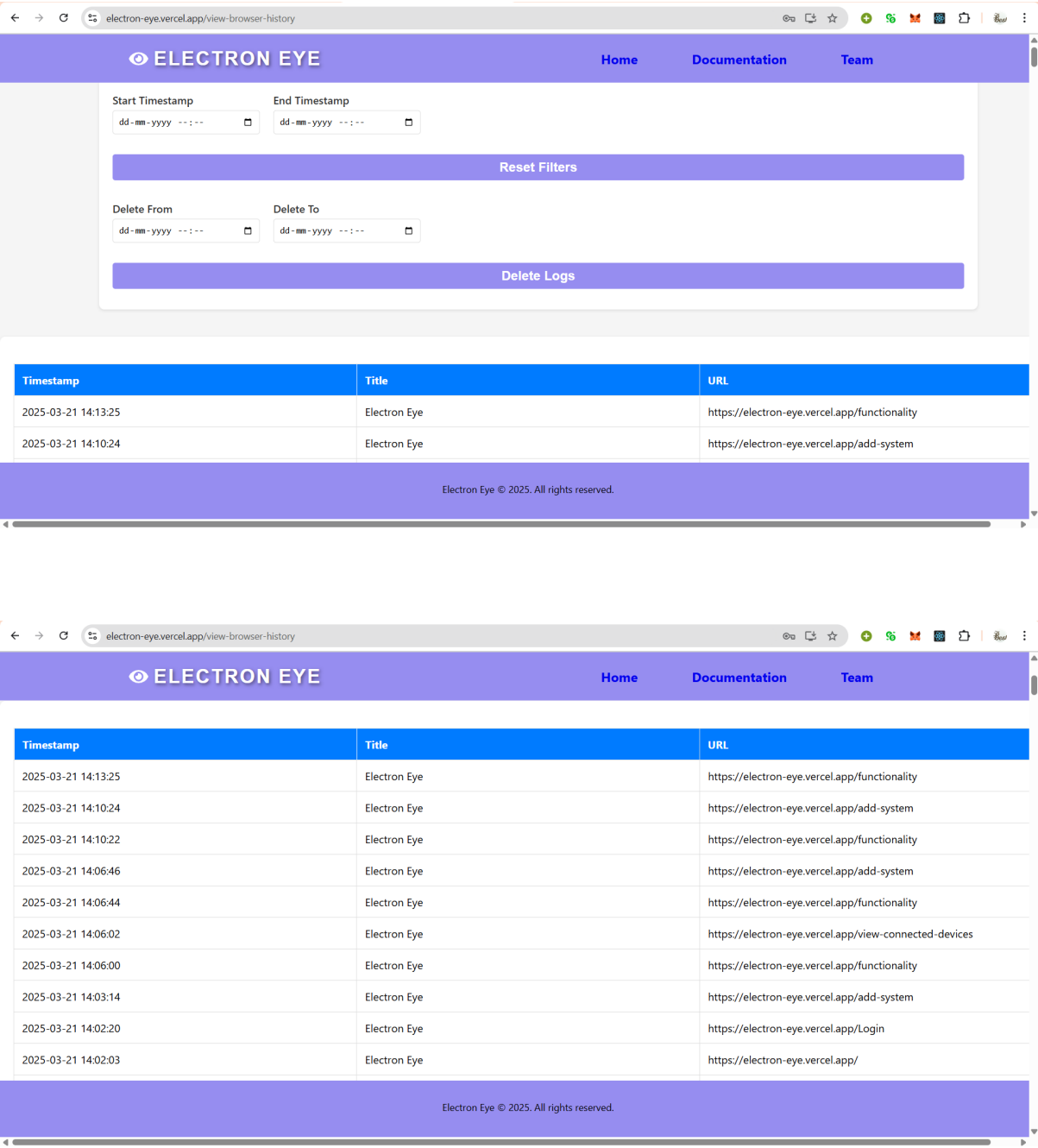


Figure 8.13: Browser History

Electron Eye: Activity Tracking and Visualisation Application

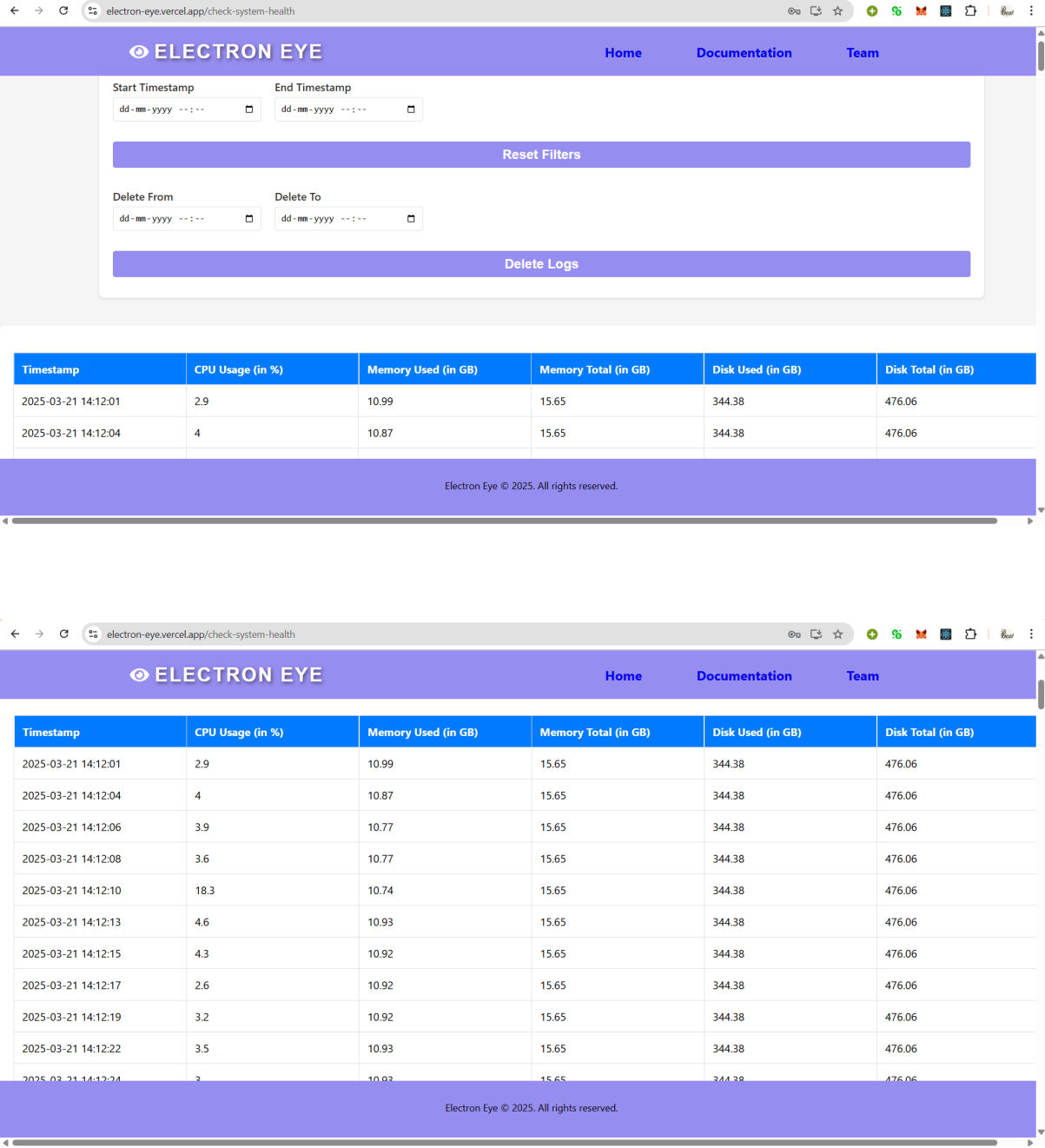


Figure 8.14: System Health

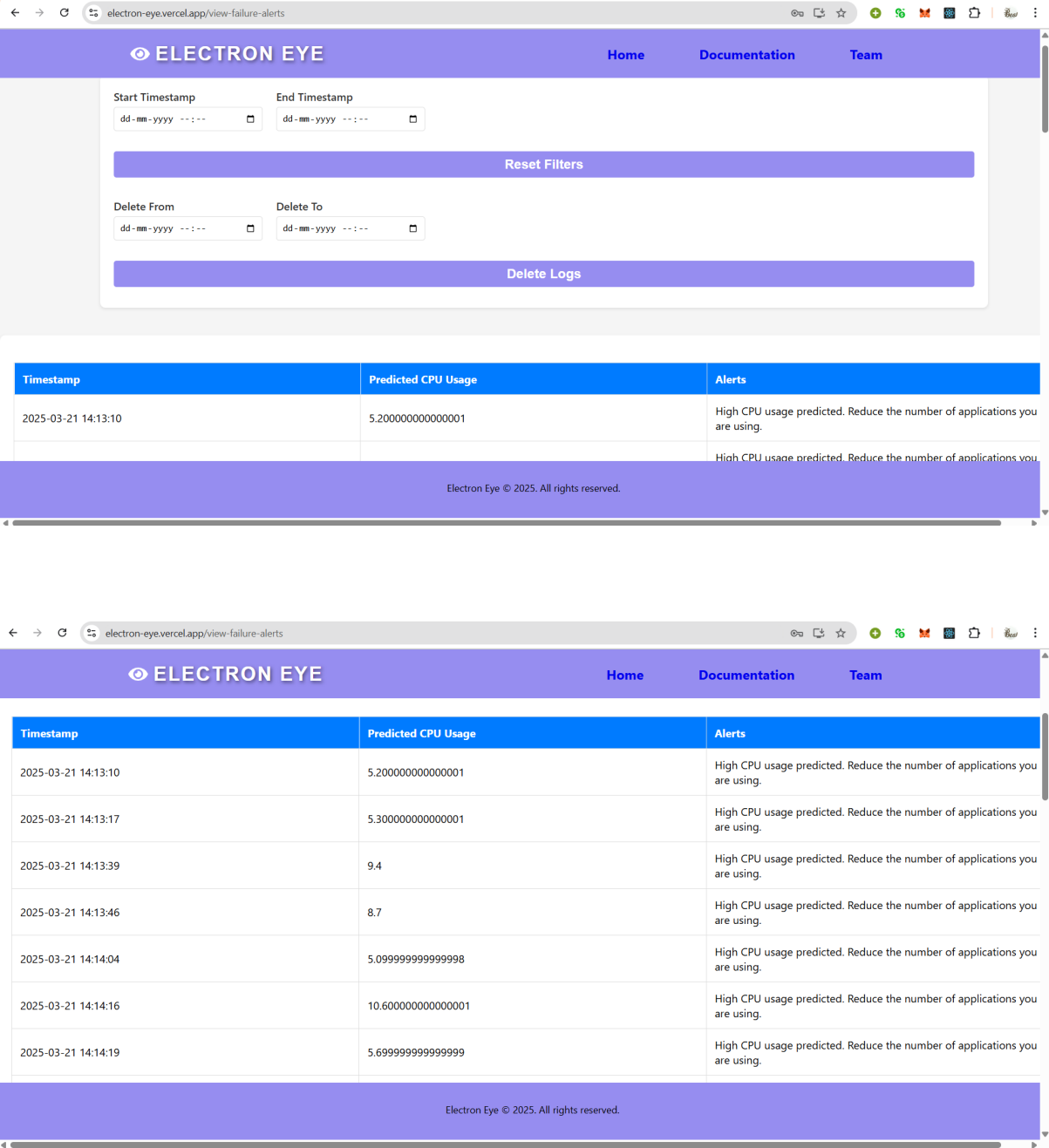


Figure 8.15: Failure Alerts

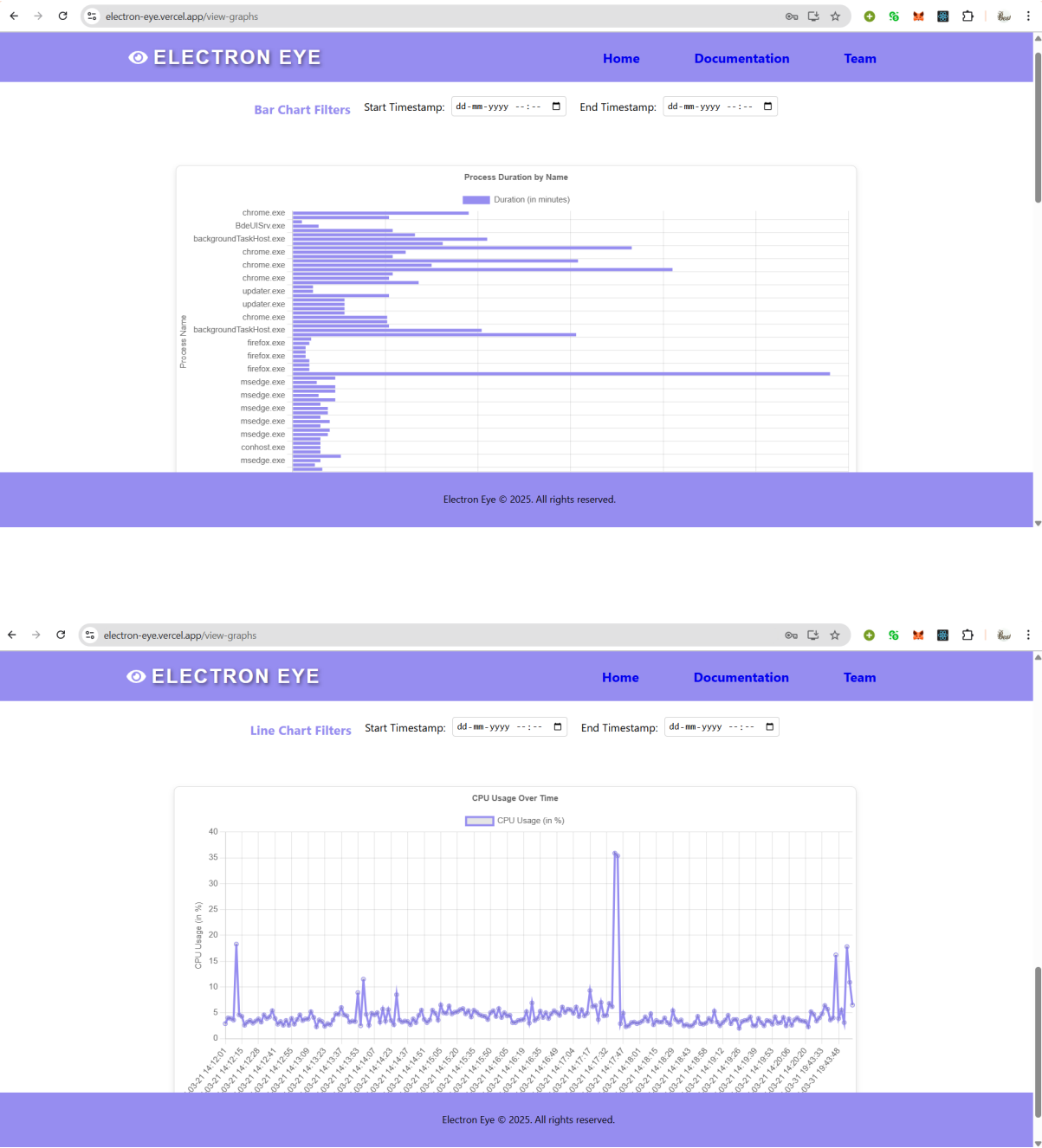


Figure 8.16: Graphs

CHAPTER 9

CONCLUSIONS

9.1 Conclusion

The proposed lab management desktop application will offer a cutting-edge solution that addresses the limitations of current systems. It will provide cross-platform support, allowing seamless integration across various operating systems, and will aggregate real-time data from multiple computers to a central server for unified monitoring. Through its predictive maintenance features, the system will enable administrators to foresee potential system failures and take proactive measures to maintain optimal lab performance.

With advanced visualization and reporting tools, administrators will be able to identify trends, optimize resource allocation, and make data-driven decisions. The system's user-friendly interface will ensure that lab management tasks are handled efficiently, even by those with limited technical expertise. As the lab grows, the application will scale to accommodate increasing data volumes and evolving needs, making it a valuable, long-term solution for efficient and effective lab management.

9.2 Future Work

- **Scalability and Flexibility:** The current system is tailored to specific lab environments. Future versions could focus on enhancing scalability and flexibility by incorporating features that cater to different lab sizes, configurations, and industry requirements, making the application adaptable for a wider range of users.
- **Integration with Cloud Services:** To enhance accessibility and data storage, future development could integrate cloud-based solutions for data management. This would allow administrators to access and manage lab data remotely, ensuring system performance and health monitoring beyond local networks.
- **Enhanced Predictive Maintenance Capabilities:** While the current system provides predictive maintenance, future updates could incorporate advanced machine learning algorithms. These algorithms would analyze historical data more effectively to predict system malfunctions or potential issues, improving maintenance accuracy and reducing downtime further.
- **User Training and Onboarding Modules:** Introducing training modules and onboarding guides for lab managers could improve the user experience and efficiency. These modules could provide real-time guidance and instructions on how to use the software effectively, reducing the learning curve for new users.
- **Mobile Application Development:** Developing a companion mobile application could enable administrators to monitor and manage lab activities on the go. This app could provide real-time notifications, system health alerts, and access to critical reports, enhancing the convenience and efficiency of lab management.

9.3 Applications

- **University Computer Labs:** In educational institutions, this system can be deployed in computer labs to monitor and manage computer usage, ensuring that

resources are used efficiently. Lab administrators can use the predictive maintenance feature to minimize downtime and maintain system performance, allowing students and faculty to have reliable access to lab resources for their academic and research needs.

- **Corporate Training Centers:** For businesses with in-house training facilities or labs, this application can monitor and optimize computer usage. The analytics and reporting features can help administrators understand usage patterns, allowing them to adjust schedules and resources accordingly. Predictive maintenance ensures that systems are operational during critical training sessions, improving overall training effectiveness and ensuring that learning environments are continuously available.
- **Healthcare Simulation Labs:** In medical training facilities, this system helps manage simulation labs where trainees use specialized software for practice. By monitoring system usage and health, lab managers can ensure that equipment is functional and ready for use. Predictive maintenance helps prevent unexpected failures, ensuring continuous operation of simulation tools that are vital for hands-on medical training and education.
- **Research and Development Facilities:** R&D labs, especially those involving high-performance computing, can benefit from this application by tracking system performance and usage. Lab administrators gain insights into how systems are being utilized, enabling them to optimize resource allocation for various projects. The predictive maintenance feature ensures that critical research tasks are not interrupted by system failures, enhancing overall research productivity.
- **Public Libraries and Community Centers:** This system can be used to manage and maintain computer resources in public libraries or community centers where multiple users access systems daily. By tracking usage patterns, administrators can determine peak usage times and allocate resources accordingly. The predictive maintenance feature allows for proactive system management, reducing the risk of computer downtime, which is essential for maintaining public access to technology and information.

CHAPTER 10

APPENDIX

10.1 Appendix A: Problem Statement Feasibility Assessment

The proposed system aims to develop a real-time activity tracking and visualization tool for laboratory environments. To assess its feasibility:

- **Satisfiability:** The problem is satisfiable as it is possible to implement a real-time monitoring and alerting system using current technologies such as Python, Node.js, React, and MongoDB.
- **Complexity Classification:**
 - The system deals with log collection, pattern identification, and alert generation.
 - These are computationally solvable in polynomial time (**P-Type**), as no known NP-complete reductions or exponential search spaces are involved in real-time monitoring tasks.
- **Mathematical Representation:** The system is modeled as:

$$S = \{I, P, O, F, D, A, R\}$$

where each element denotes inputs, processes, outputs, functions, database, actors, and rules respectively (as defined in the Mathematical Model section).

- **Modern Algebra Feasibility:** The modules and transitions can be visualized using a Finite State Machine (FSM) or function mappings:

$$f : I \rightarrow O \quad (\text{functions map inputs to outputs with defined rules})$$

making the system functionally complete and logically implementable.

Hence, the proposed problem is feasible, solvable in polynomial time, and does not fall under NP-Complete or NP-Hard categories.

10.2 Appendix B: Paper Publication Details

- **Title:** Electron Eye: Activity Tracking and Visualisation Application
- **Name of Conference:** 2025 1st Global Conference on Cognitive Computing and Communication Technology (GC4T) 2025
- **Reviewer's Comments:**
 - “Well-defined scope and clearly structured architecture.”
 - “Real-time alerting mechanism is practical for institutional environments.”
 - “Could further explore the scalability and anomaly detection components.”
 - “The paper introduces a unique solution tailored for lab management, integrating predictive maintenance, system monitoring, and visualization.”
 - “Cross-platform compatibility and centralized data aggregation make the approach distinctive.”
 - “Visualization using real-time dashboards adds value for decision-making by lab managers.”
 - “Plagiarism check is recommended to confirm originality.”
 - “Needs more clarity on dataset size, type, and performance validation under scale.”
 - “Data privacy and security measures should be discussed in more depth.”
 - “Comparative analysis with existing lab monitoring tools would strengthen the contribution.”
- **Status:** Presented

10.3 Appendix C: Plagiarism Report

Tool Used: <https://plagiarismdetector.net/>

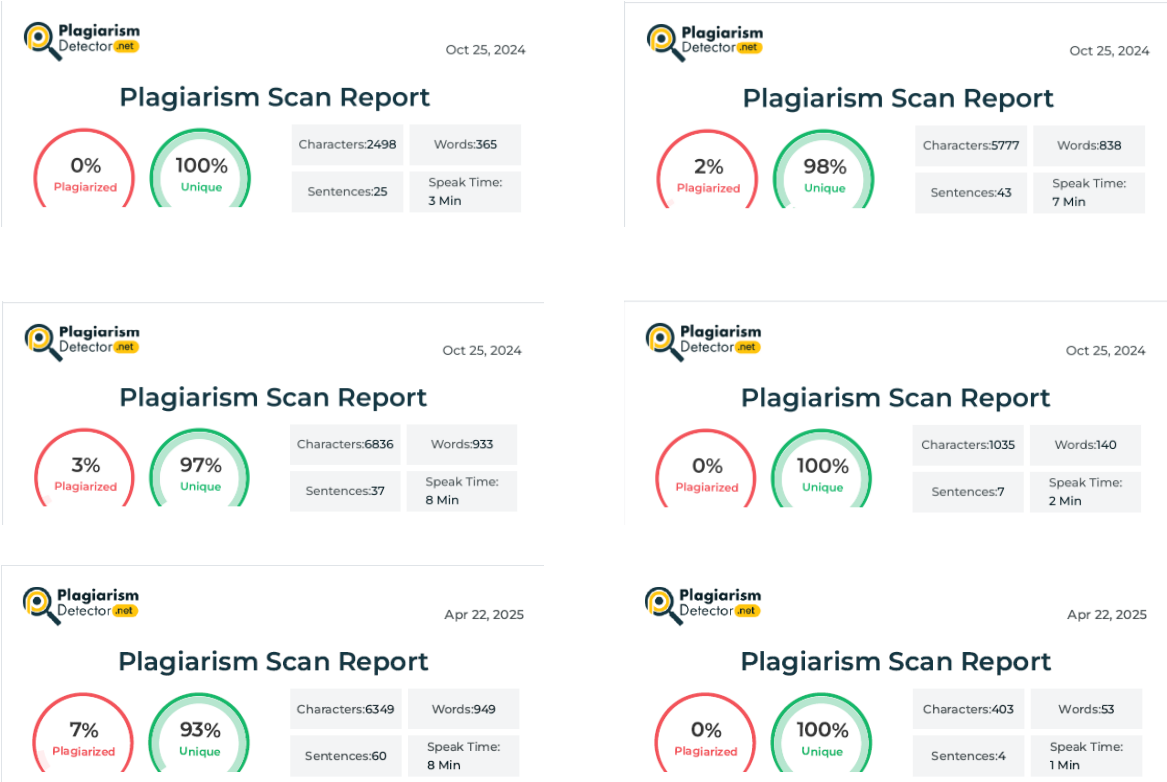


Figure 10.1: Plagiarism Report Snapshots for Various Project Sections

Thus, the overall plagiarism percentage is **2%**.

CHAPTER 11

REFERENCES

- [1] Sharma, Hitesh. (2019). Real Time Activity Logger: a User Activity Detection System. *International Journal of Engineering and Advanced Technology*. 9. 10.35940/ijeat.A9529.109119.
- [2] A. Keane and S. O'Shaughnessy, "Tracking User Activity on Personal Computers," in *Proceedings of the 2011 International Conference*, pp. 188-196, 2011, doi: 10.1007/978-3-642-35515-8_16.
- [3] S. A., "Collaborative Time Tracking System for Employees," *IJARCCCE*, vol. 7, pp. 78-86, 2018, doi: 10.17148/IJARCCCE.2018.71117.
- [4] K. Zhang, J. Xu, M. Min, G. Jiang, K. Pelechrinis, and H. Zhang, "Automated IT system failure prediction: A deep learning approach," in *2016 IEEE International Conference on Big Data*, pp. 1291-1300, 2016, doi: 10.1109/BigData.2016.7840733.
- [5] P. Yoosook and P. Apirukvorapinit, "Performance monitoring tool for mobile application," *2018 5th International Conference on Business and Industrial Research (ICBIR)*, Bangkok, Thailand, 2018, pp. 177-182, doi: 10.1109/ICBIR.2018.8391188.
- [6] M. Sahasrabudhe, M. Panwar and S. Chaudhari, "Application performance monitoring and prediction," *2013 IEEE International Conference on Signal Processing, Computing and Control (ISPCC)*, Solan, India, 2013, pp. 1-6, doi: 10.1109/IS-PCC.2013.6663466.
- [7] E. Fulp, G. Fink, and J. Haack, "Predicting Computer System Failures Using Support Vector Machines," 2008.
- [8] V. Lobo and N. Ansari, "Web-based Automated Time and Effort Tracking Software for a Software Project," 2016.
- [9] S. Tiwari, T. Tiwari, and T. Tiwari, "A secured MAC address based Login System," *European Journal of Electrical Engineering and Computer Science*, vol. 2, no. 2, 2018, doi: 10.24018/ejece.2018.2.2.16.
- [10] M. Cengiz, M. Forshaw, A. Atapour-Abarghouei, and A. S. McGough, "Predicting the Performance of a Computing System with Deep Networks," in *Proceedings of the 2023 ACM/SPEC International Conference on Performance Engineering (ICPE '23)*, Association for Computing Machinery, New York, NY, USA, pp. 91-98, 2023, doi: 10.1145/3578244.3583731.
- [11] D. Srivastava, "An Introduction to Data Visualization Tools and Techniques in Various Domains," *International Journal of Computer Trends and Technology*, vol. 71, no. 4, pp. 125-130, 2023, doi: 10.14445/22312803/IJCTT-V71I4P116.
- [12] A. Lavanya, S. Sindhuja, L. Gaurav, and W. Ali, "A Comprehensive Review of Data Visualization Tools: Features, Strengths, and Weaknesses," *Int. J. Comput. Eng. Res. Trends*, vol. 10, no. 1, pp. 10-20, Jan. 2023.
- [13] R. Vilalta, C. Apte, J. Hellerstein, S. Ma, and S. Weiss, "Predictive algorithms in the management of computer systems," *IBM Systems Journal*, vol. 41, pp. 461-474, 2002, doi: 10.1147/sj.413.0461.

- [14] R. Gunasekaran, S. Oral, D. Dillow, B. Park, G. Shipman, and Oak Ridge National Laboratory, "Real-Time System Log Monitoring/Analytics Framework," 2012. [Online]. Available: https://cug.org/5-publications/proceedings_attendee_lists/CUG11CD/pages/1-program/finalprogram/Monday/04B-Gunasekaran-Paper.pdf

LIST OF TABLES

Table	Illustration	Page No.
3.1	User Interfaces	10
3.2	Hardware Interfaces	10
3.3	Software Interfaces	10